深圳河套學院
Shenzhen Loop Area Institute

Natural Language Processing and Large Language Models

Xiaojun Quan and Zhengjun Yue

Shenzhen Loop Area Institute

Autumn, 2026

Lecture 4

Attention and Transformer

Learning Objectives

By the end of this section, you should be able to:

1. Explain why *attention* is needed and how it helps a model use relevant information from different positions.
2. Describe the core mechanism of self-attention, including *queries*, *keys*, *values*, and *attention weights*.
3. Understand how *Transformer* blocks combine *self-attention*, *feed-forward layers*, *residual connections*, and *positional information*.
4. Explain why *Transformers* are better suited than RNNs for long-range dependencies and parallel computation.

Outline

- Recap
- Self-Attention
- Position Embeddings
- Masked Self-Attention
- Transformer
- Practice & Discussion

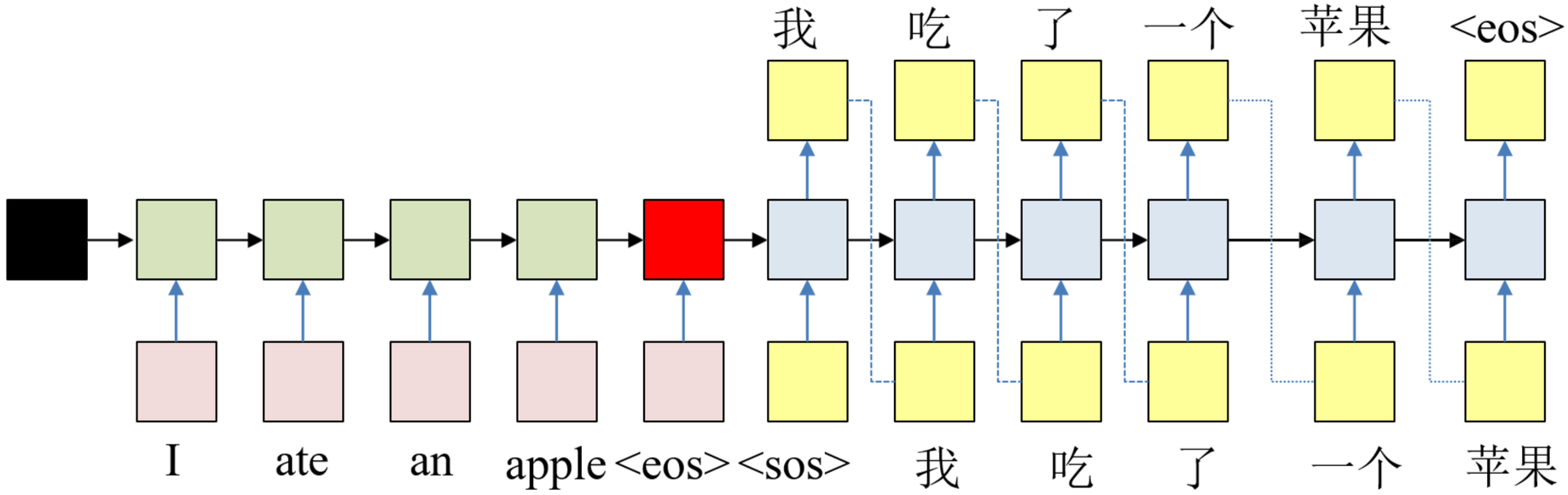
Outline

- **Recap**
- Self-Attention
- Position Embeddings
- Masked Self-Attention
- Transformer
- Practice & Discussion

Recap: Recurrent Neural Networks

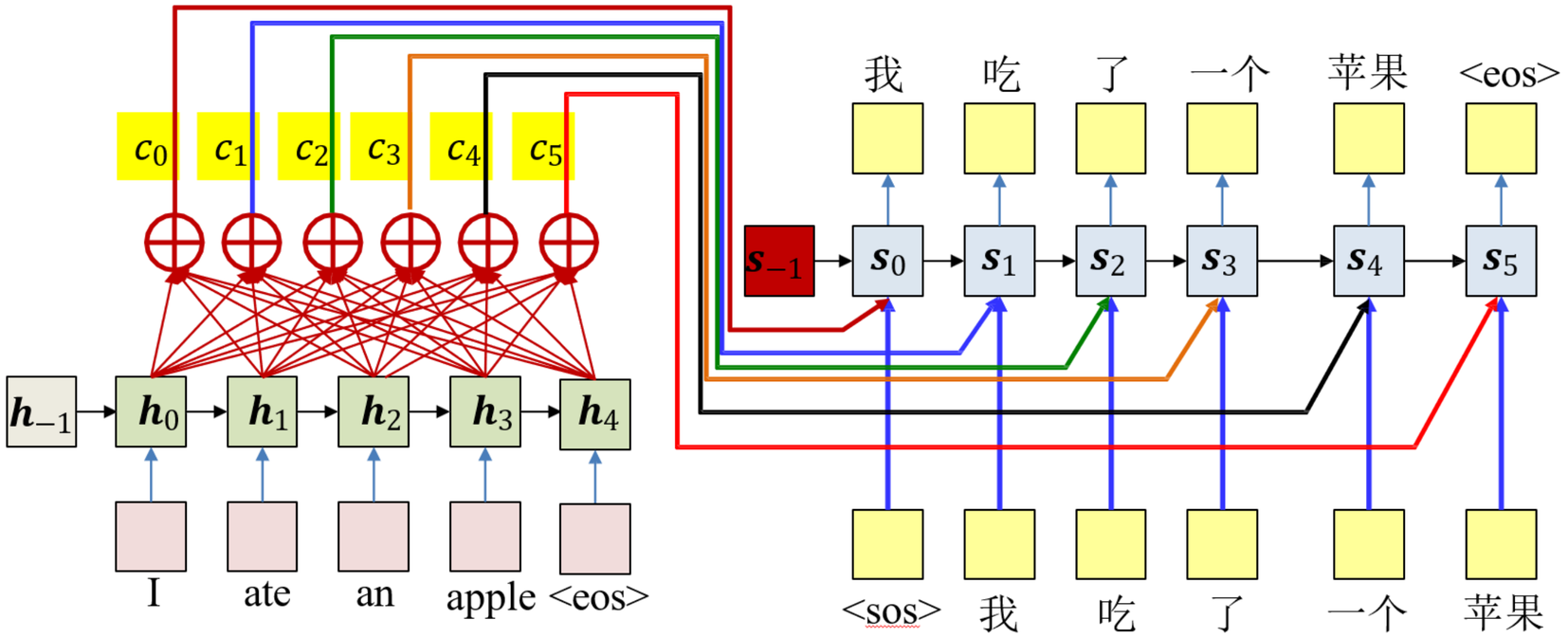
- Issues with RNNs:
  - Longer sequences can lead to vanishing gradients → hard to capture long-distance information.
  - Lack parallelizability (并行)
    - Forward pass requires *O*(sequence length) unparallelizable operations.
    - GPUs can run many independent computations at once, but RNNs don't use that.
    - Limits training on very large datasets.
  - Limited memory
    - Past information is compressed into a single fixed-size hidden state, which cannot effectively retain long-term dependencies.

Recap: Seq2Seq Models



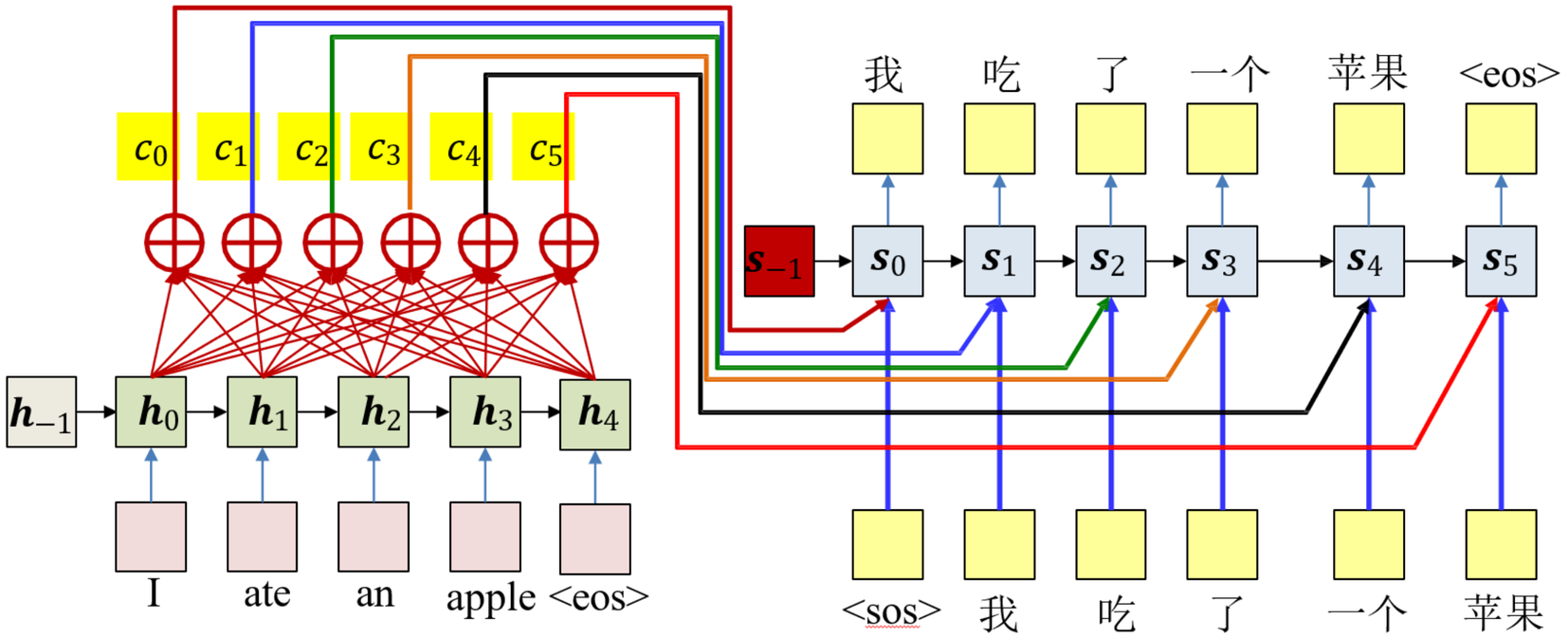
- The input sequence feeds into a *recurrent* structure.
- The input sequence is terminated by an <EOS> symbol.
  - The hidden state at the <EOS> "stores" all information about the sentence.
- *Another RNN* takes the hidden state as initial state to produce output.

Recap: Attention Models



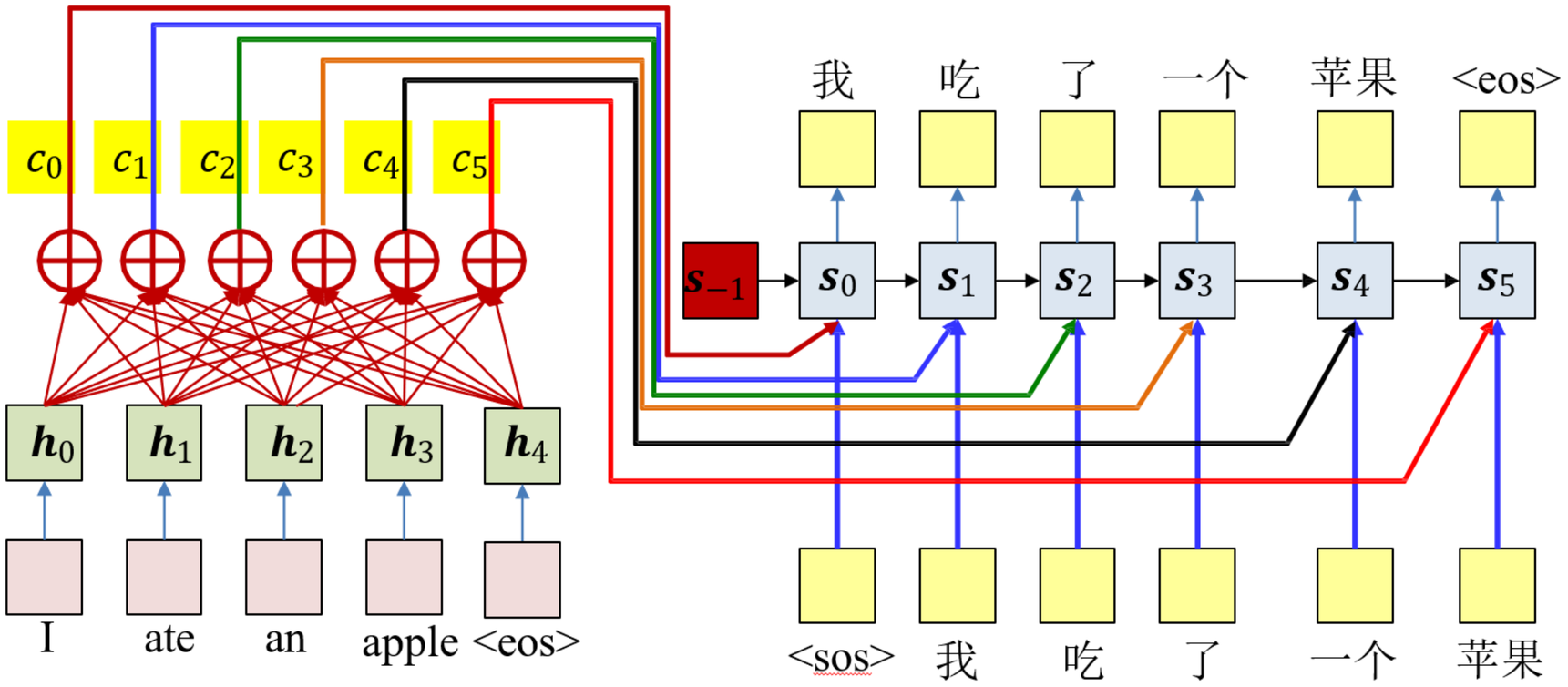
- **Encoder** recurrently produces hidden representations of input sequence.
- **Decoder** recurrently generates output word sequence.
  - For each output word, the decoder uses a weighted average of input representations as context, plus the previous hidden state and output word.

Recap: Attention Models



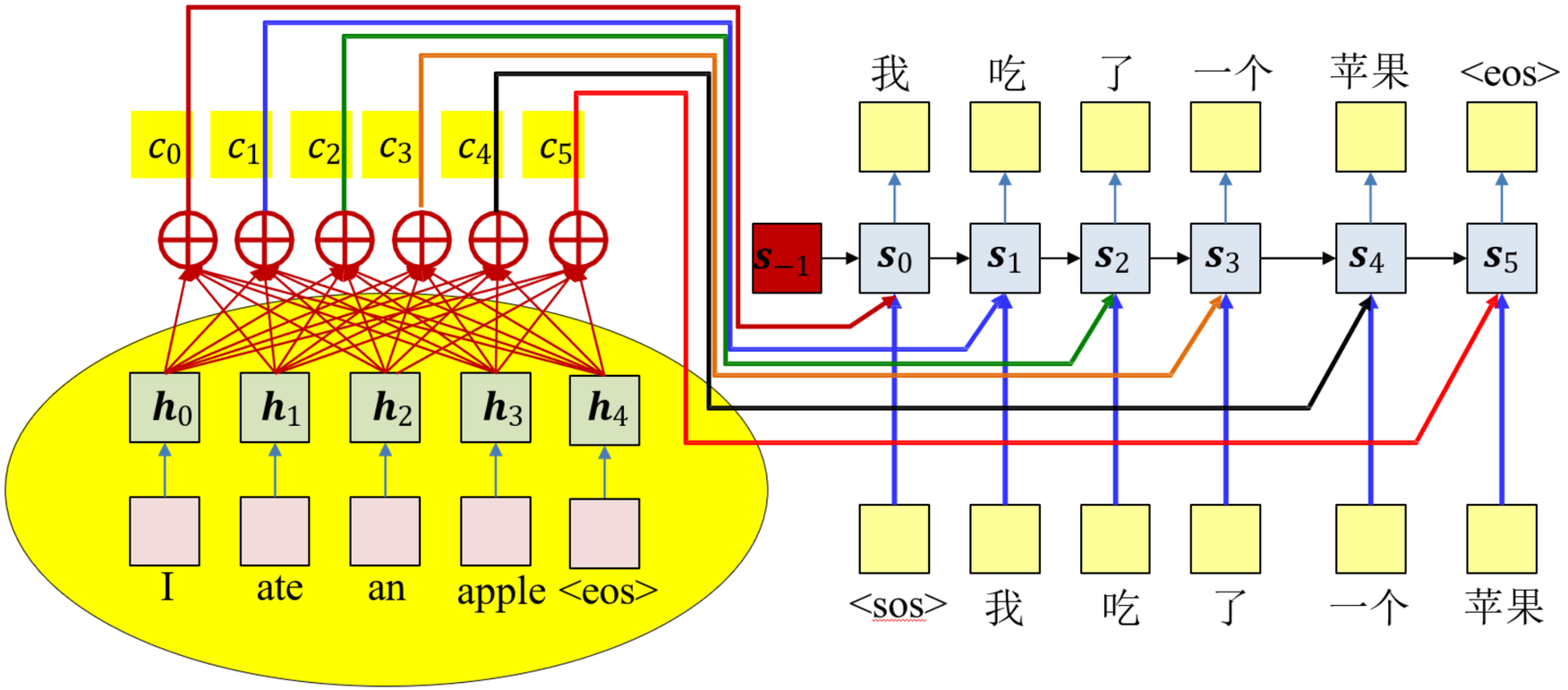
- **Problem**: Because of the *recurrence*, the hidden representation for any word is also influenced by all preceding words.
  - Actually attend to a *sequence representation*, rather than a single word.
- If the decoder is automatically figuring out *which words of the input to attend to at each time*, is *recurrence* even necessary?

Recap: Non-Recurrent Encoder



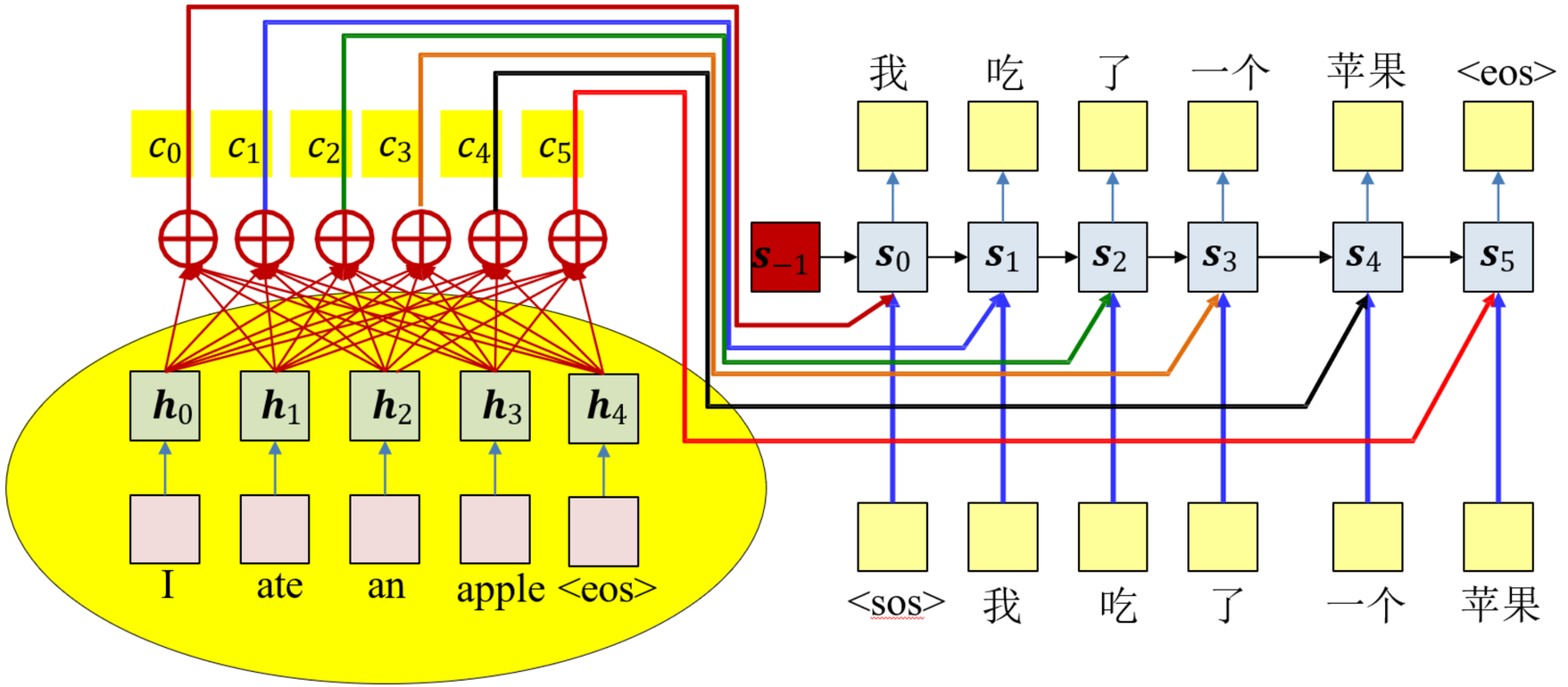
- **Modification**: Let us eliminate the *recurrence* in the encoder.

Recap: Non-Recurrent Encoder



- But this eliminates context-specificity in encoder embeddings.
  - The embedding for "*apple*" depends on the remaining words.
- **Solution**: Use the attention method to introduce context-aware embeddings.

Recap: Non-Recurrent Encoder



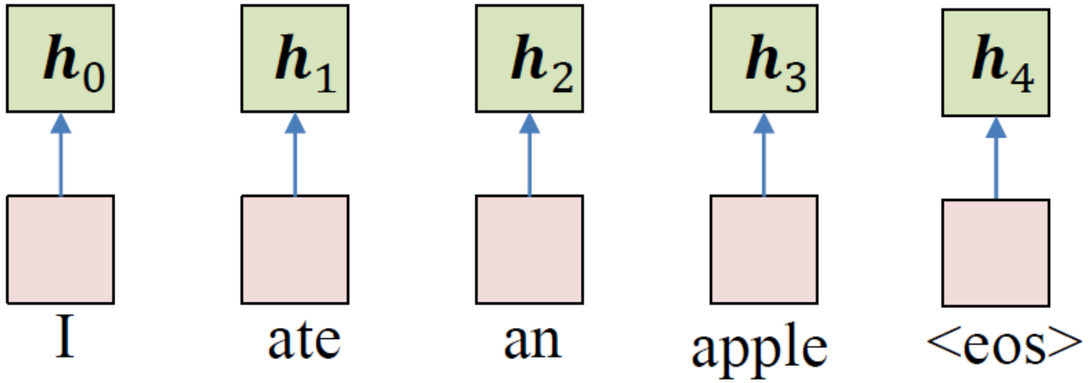
- The encoder in a seq-to-seq model can be developed without *recurrence*.
- Use the attention method itself to introduce context-aware embeddings.
  - "Self" attention (自注意力).

Outline

- Recap
- **Self-Attention**
- Position Embeddings
- Masked Self-Attention
- Transformer
- Practice & Discussion

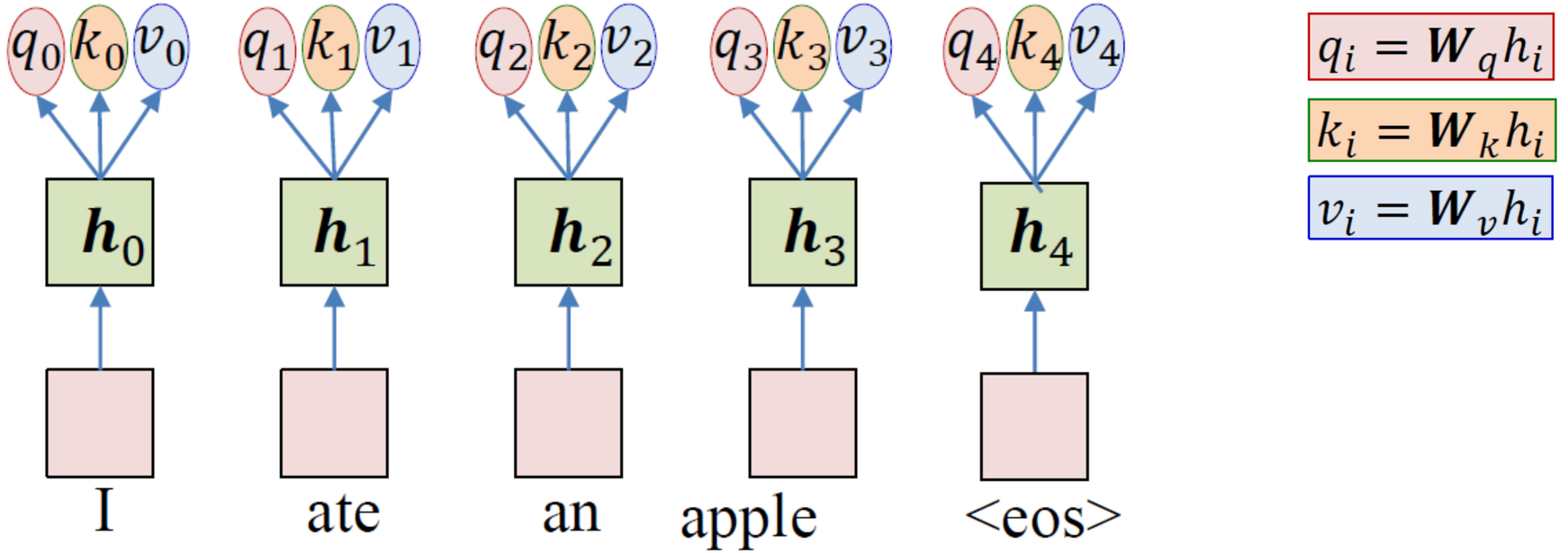
Self-Attention

- **First**, for every word in the input, compute an initial representation.
  - E.g., using a *single MLP layer*.



Self-Attention

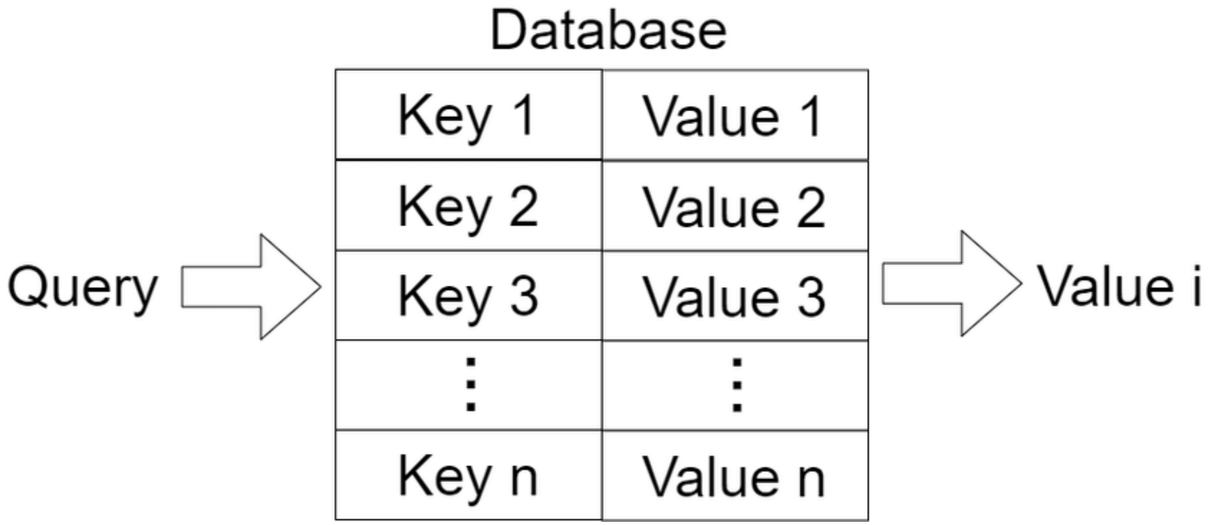
- **Then**, from *each hidden state*, compute a *query*, a *key*, and a *value*.
  - Using separate *linear transforms*.
  - The weight matrices W_q, W_k and W_v are learnable parameters.



Self-Attention

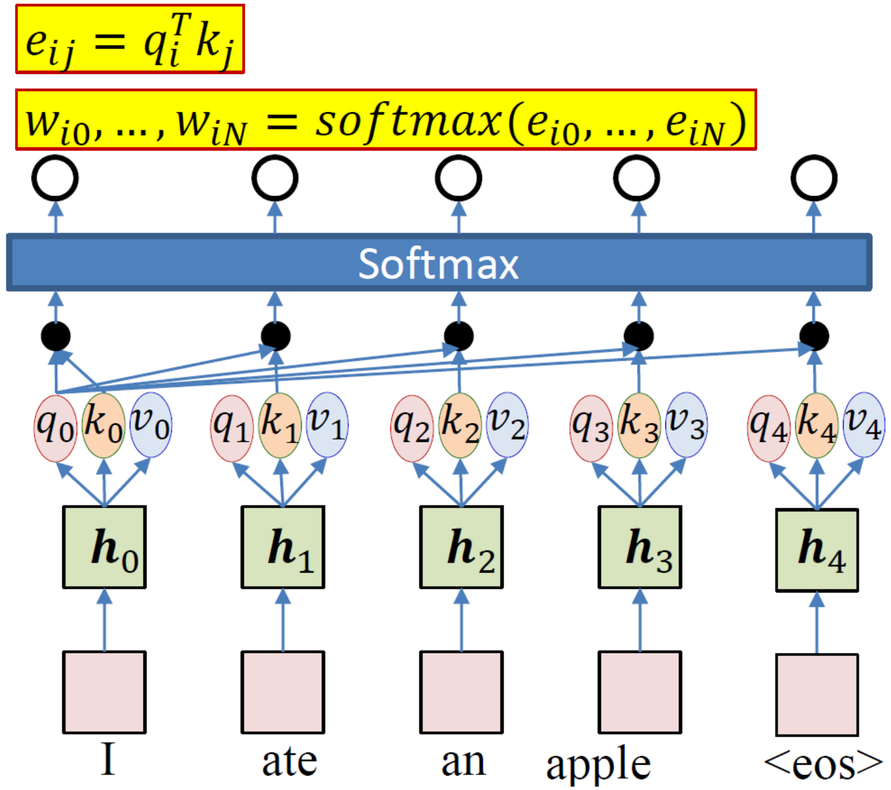
Question: Why does the Transformer compute separate Query, Key, and Value vectors instead of using the original hidden state directly in self-attention?

In essence: Q, K, V are learned transformations of the hidden state that let attention be expressive and directional, whereas the raw hidden state alone can't differentiate these roles.



Self-Attention

- For each word, compute an attention weight between it and all other words.
  - The attention of the ith word to the jth word is a function of query q_i and key k_j.
  - Pass the attention values through a *softmax* to get the final attention weights.



$$q_i = \boldsymbol{W}_q h_i$$

$$k_i = \boldsymbol{W}_k h_i$$

$$v_i = \boldsymbol{W}_v h_i$$

$$w_{ij} = attn(q_i, k_{0:N})$$

Self-Attention

- Update representation for the word as the attention-weighted sum of the values for all words.
  - Including itself.

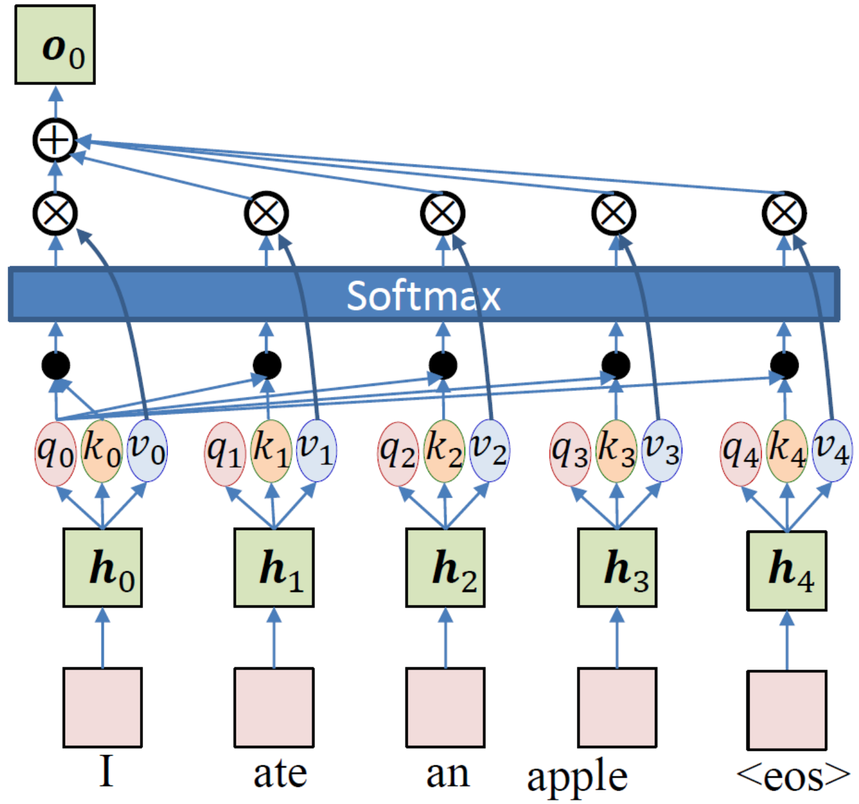
$$q_i = \boldsymbol{W}_q h_i$$

$$k_i = \boldsymbol{W}_k h_i$$

$$v_i = \boldsymbol{W}_v h_i$$

$$w_{ij} = attn(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

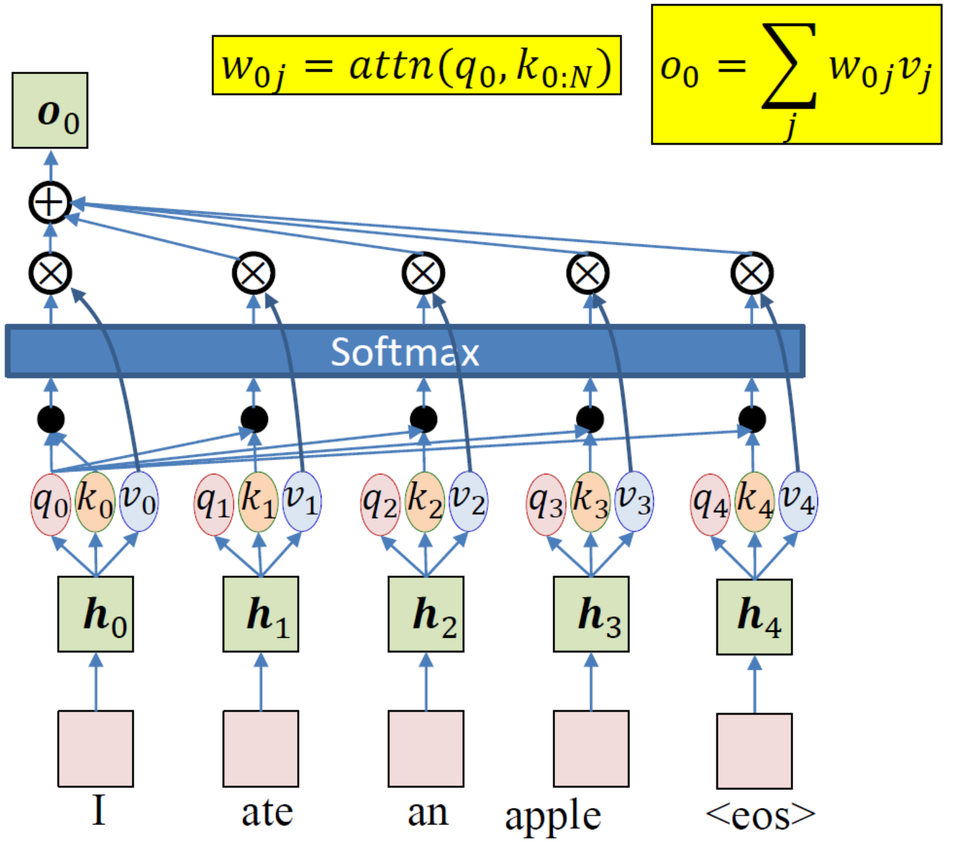


Self-Attention

$q_i = \boldsymbol{W}_q h_i$

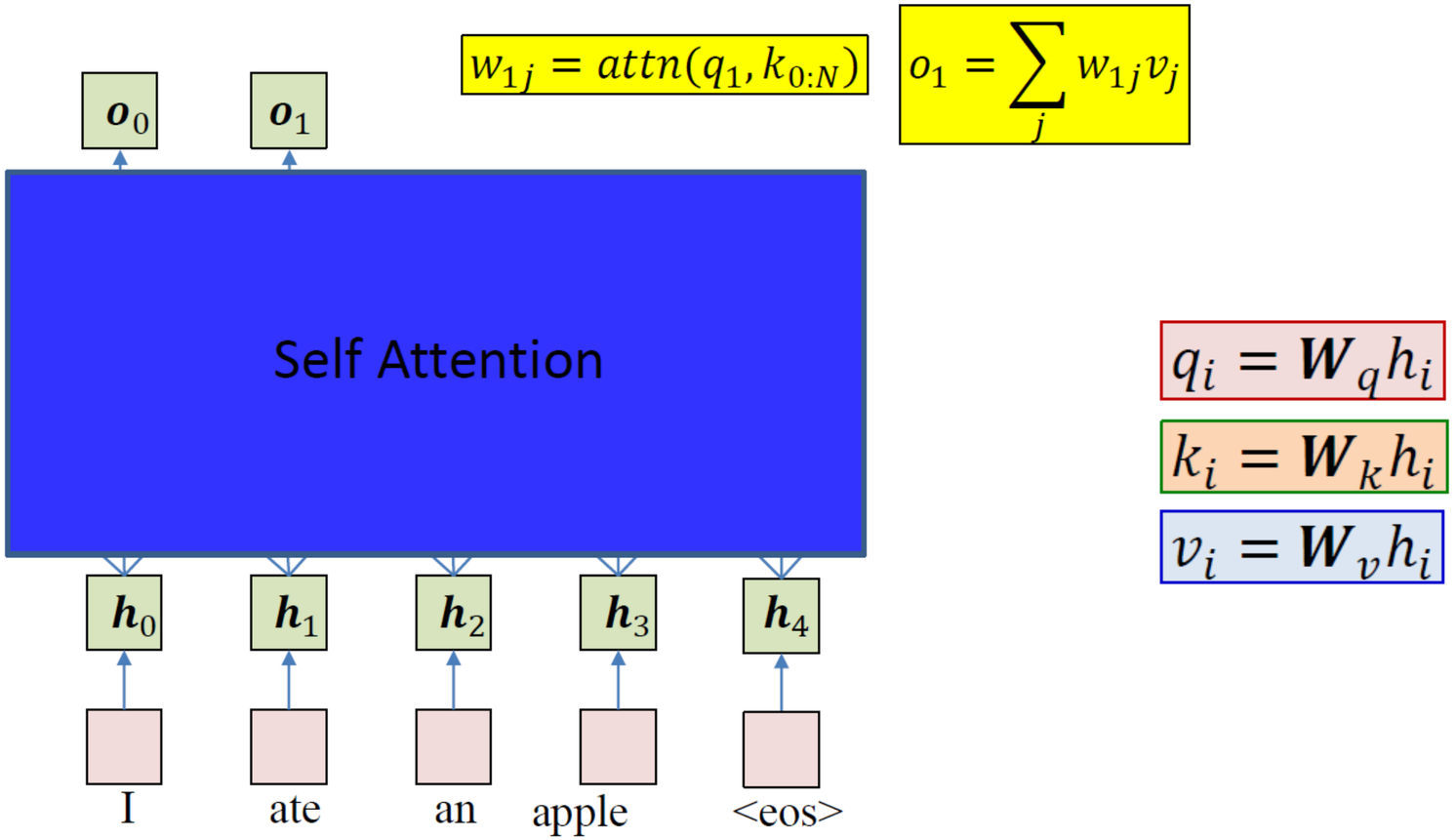
$k_i = \boldsymbol{W}_k h_i$

$v_i = \boldsymbol{W}_v h_i$



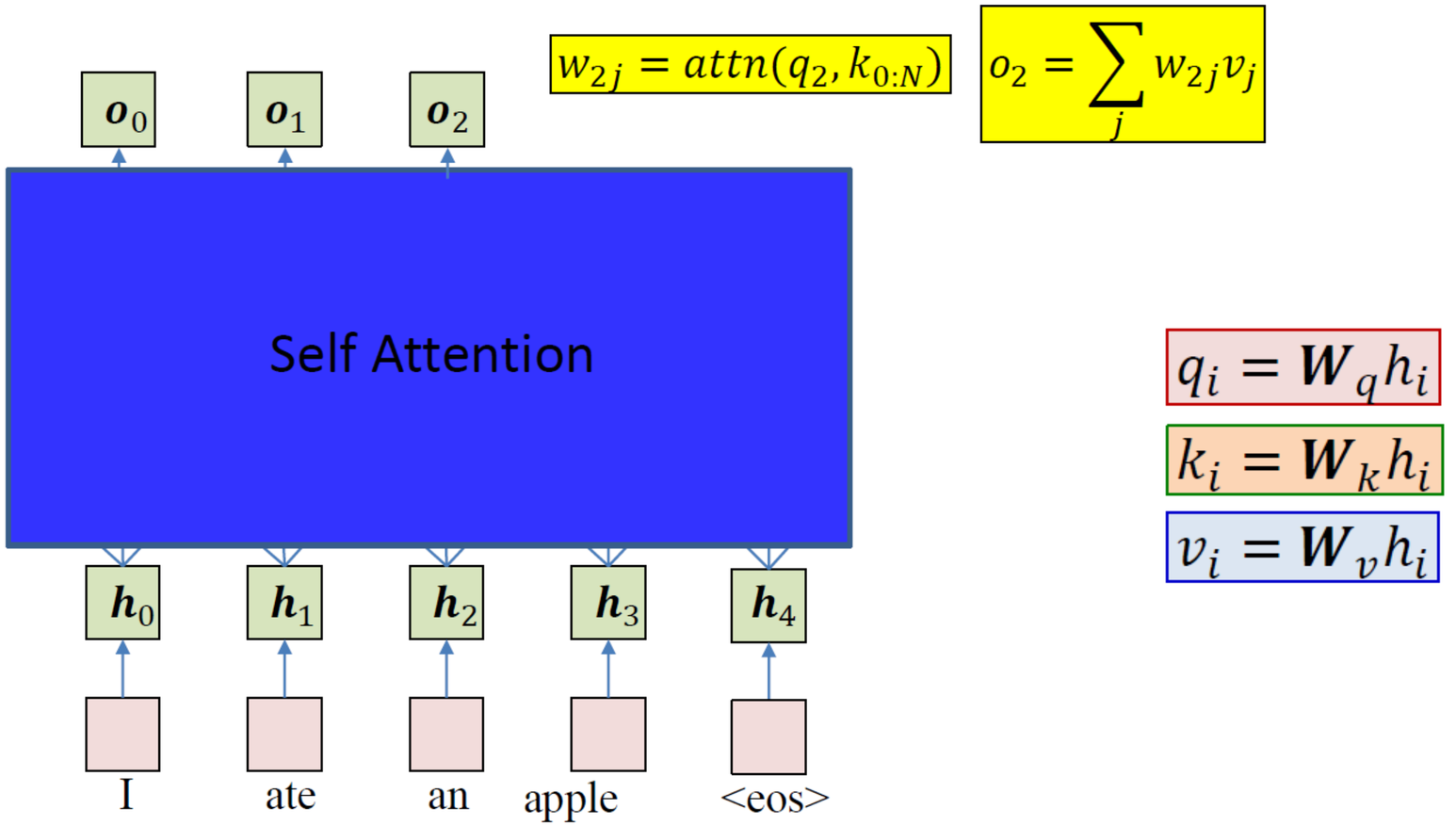
- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention



- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention



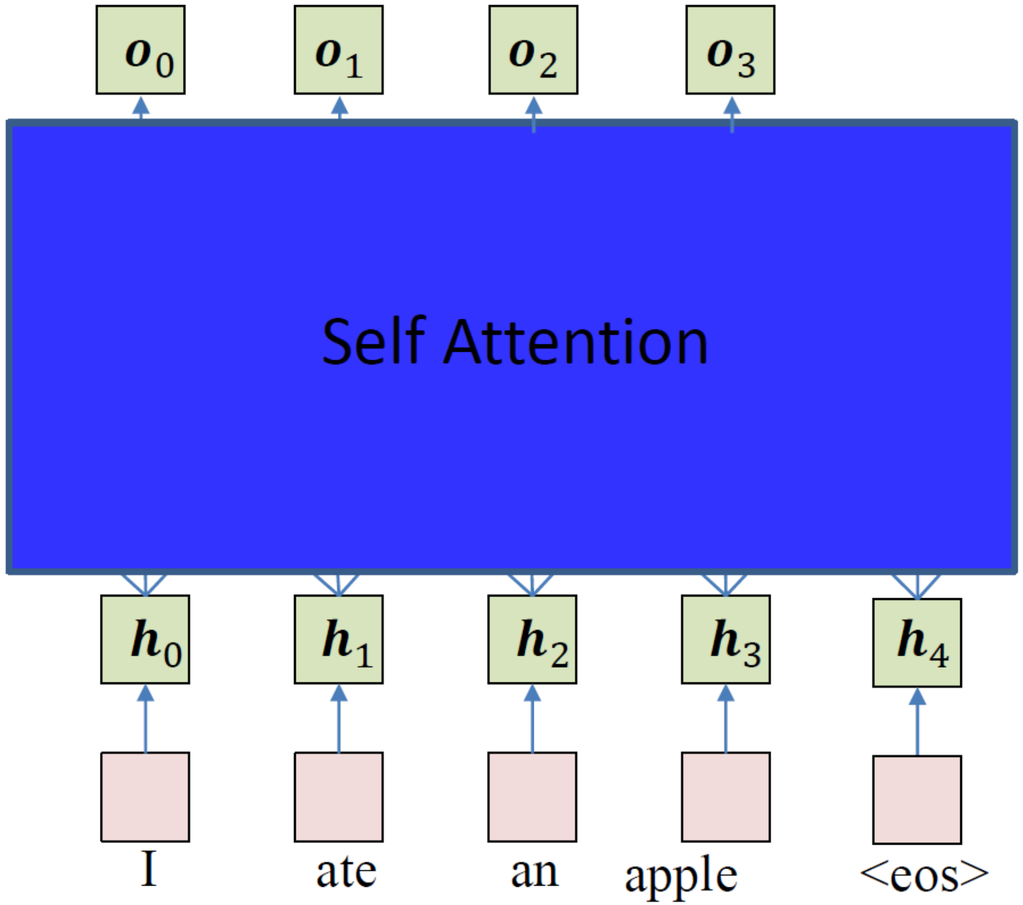
- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention

$q_i = \boldsymbol{W}_q h_i$

$k_i = \boldsymbol{W}_k h_i$

$v_i = \boldsymbol{W}_v h_i$



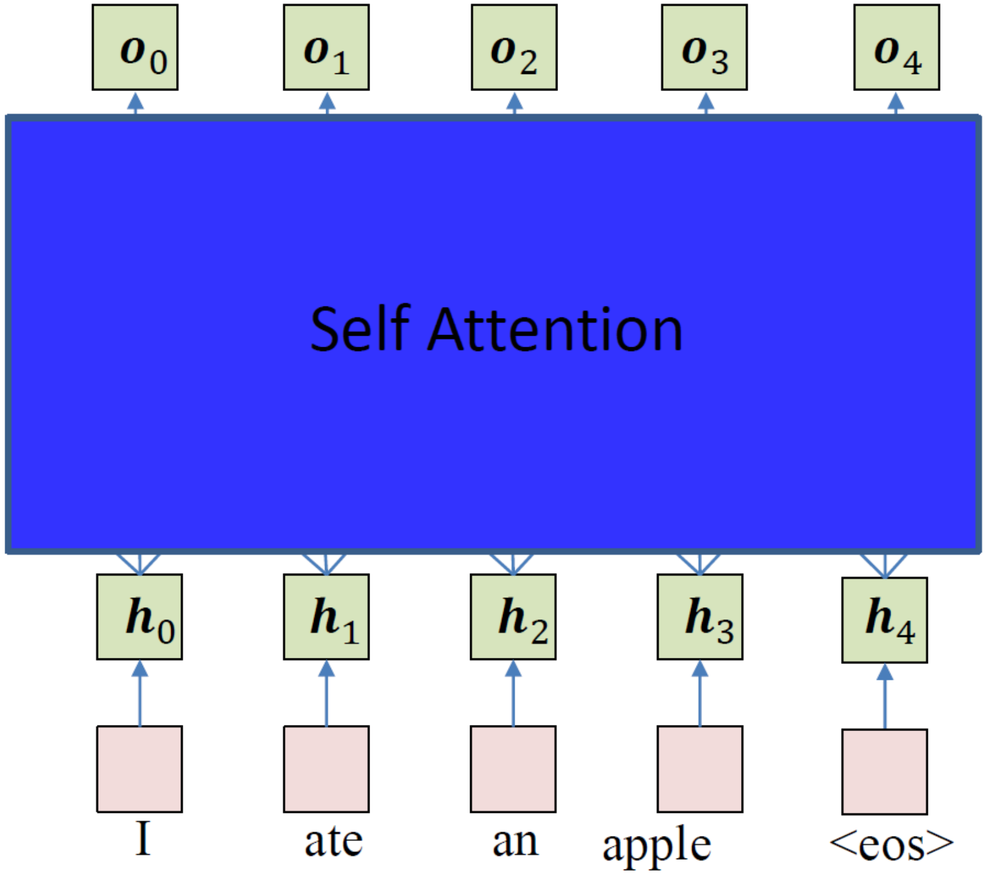
- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention

$q_i = \boldsymbol{W}_q h_i$

$k_i = \boldsymbol{W}_k h_i$

$v_i = \boldsymbol{W}_v h_i$



- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention

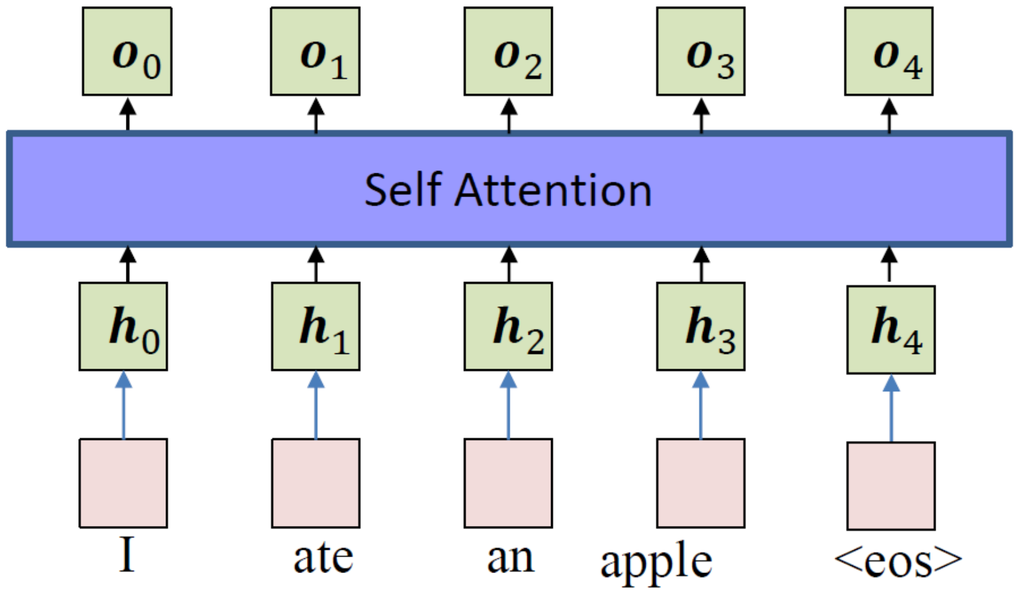
$$w_{ij} = attn(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

$$q_i = \boldsymbol{W}_q h_i$$

$$k_i = \boldsymbol{W}_k h_i$$

$$v_i = \boldsymbol{W}_v h_i$$



- Compute query-key-value sets for every word.
- For each word
  - Using its query to compute attention weights for all words using their keys.
  - Update its representation as attention-weighted sum of values of all words.

Self-Attention

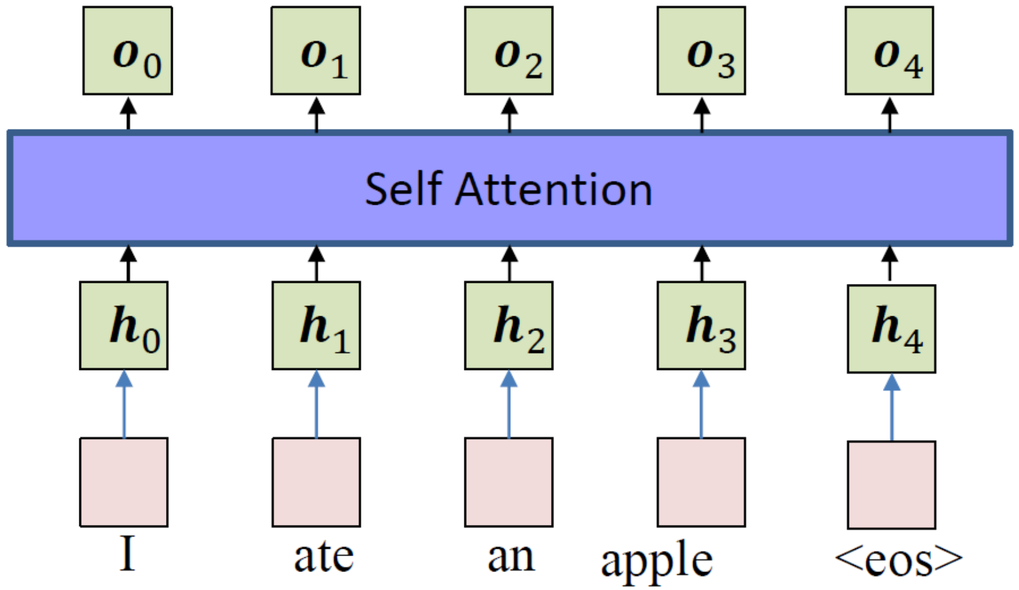
$$w_{ij} = attn(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

$$q_i = \boldsymbol{W}_q h_i$$

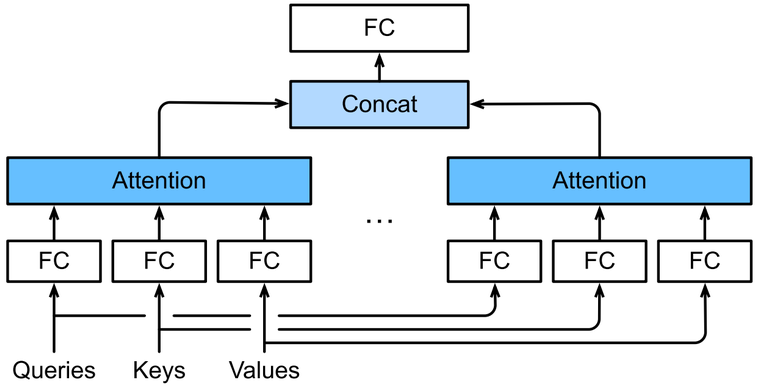
$$k_i = \boldsymbol{W}_k h_i$$

$$v_i = \boldsymbol{W}_v h_i$$



This is a “single-head” self-attention block

Multi-Head Self-Attention



- We can have multiple such attention "heads".
  - Each has independent queries, keys and values.
  - Each obtains an independent set of *attention weights*.
    - Potentially focus on a ***different aspects*** of the input.
  - Each computes an independent output.

Question: Connection between MHA with convolutional NN (CNN)?

- The final output is the concatenation of the outputs of these heads.

Vector concatenation is the process of joining two or more vectors end-to-end to create a single, larger vector. For example, concatenating vector A = [1, 2] and vector B = [3, 4] results in [1, 2, 3, 4].

Multi-Head Self-Attention

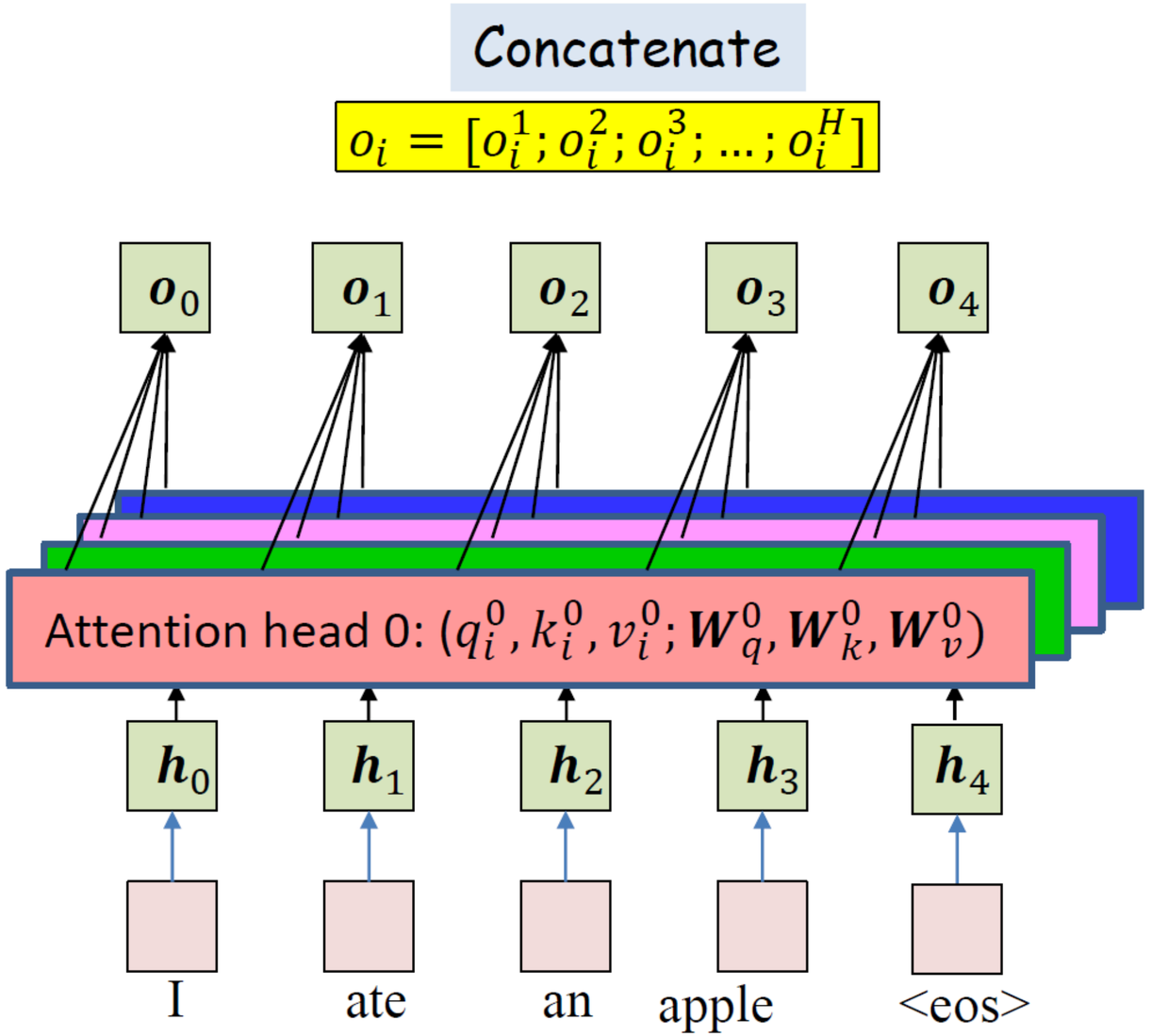
$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$



Multi-Head Self-Attention

- Multi-head self-attention
  - Multiple self-attention modules in parallel

$$q_i^a = \boldsymbol{W}_q^a h_i$$

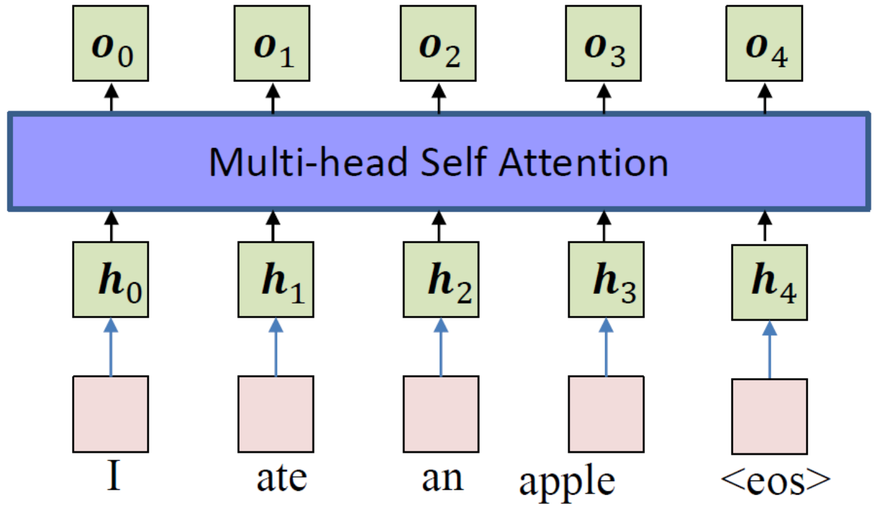
$$k_i^a = \boldsymbol{W}_k^a h_i$$

$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \ldots; o_i^H]$$



Multi-Head Self-Attention

- Typically, the output of the multi-head self attention is passed through one or more position-wise feedforward layers.
  - Followed by a non-linear activation such as *ReLU*.

$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

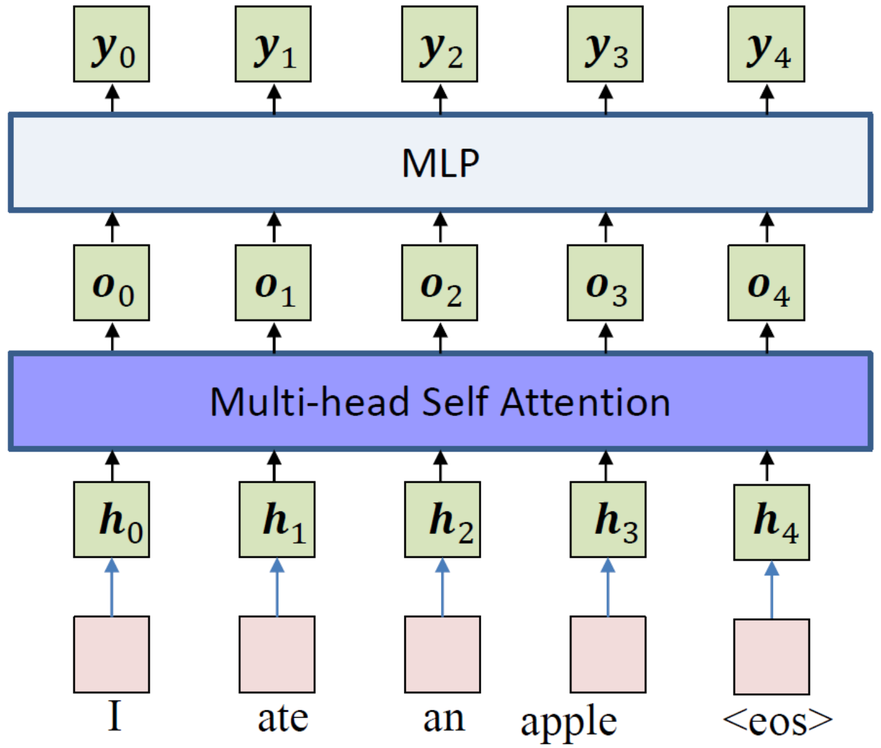
$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \ldots; o_i^H]$$

$$y_i = MLP(o_i)$$



Multi-Head Self-Attention

- The entire unit, including *multi-head self-attention* module followed by *MLP*, is a multi-head self-attention block.

$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

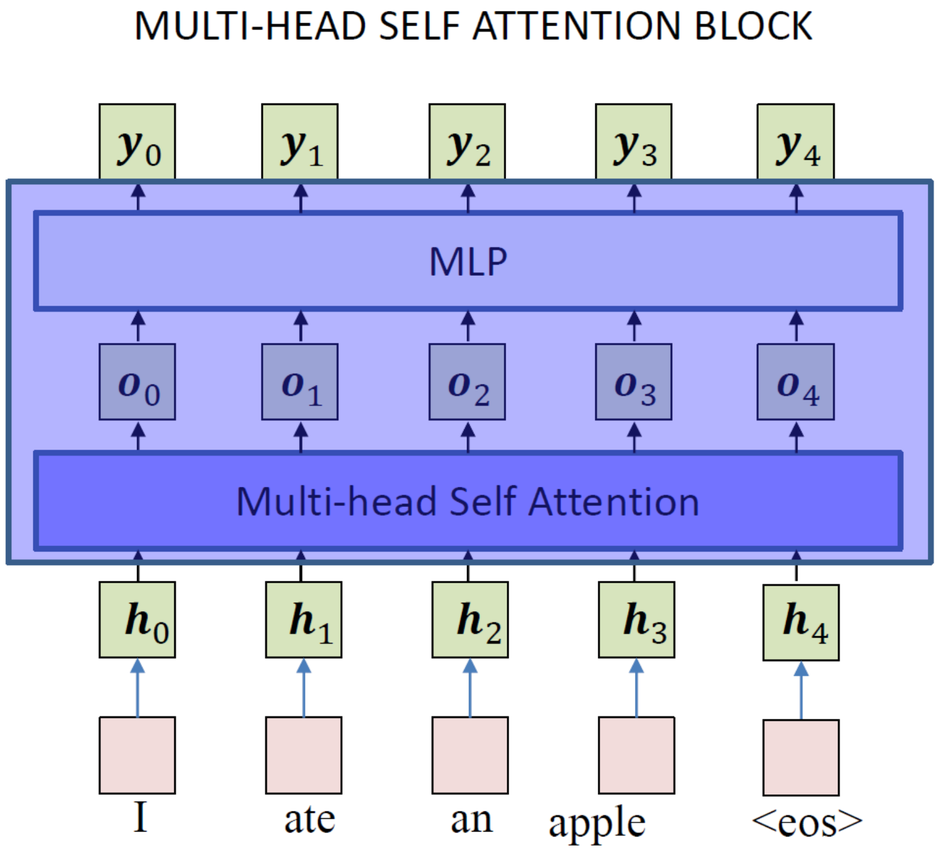
$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

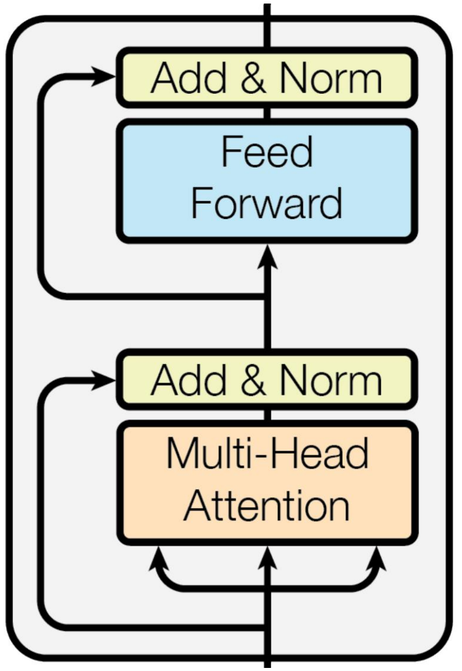
$$o_i = [o_i^1; o_i^2; o_i^3; \ldots; o_i^H]$$

$$y_i = MLP(o_i)$$



MLP (FFN) in Transformer

- What is the role of MLP (FFN) in Transformer?
  - Activation function: non-linear transformation of self-attention outputs.
  - Token-wise processing: Processes each token independently using the same learned parameters.
  - Knowledge representation: FFN parameters can encode factual and conceptual associations.



Attention mixes information across tokens, while the **FFN** transforms information within each token.

Multi-Head Self-Attention

- The entire unit, including multi-head self-attention module followed by MLP is a multi-head self-attention block.

$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

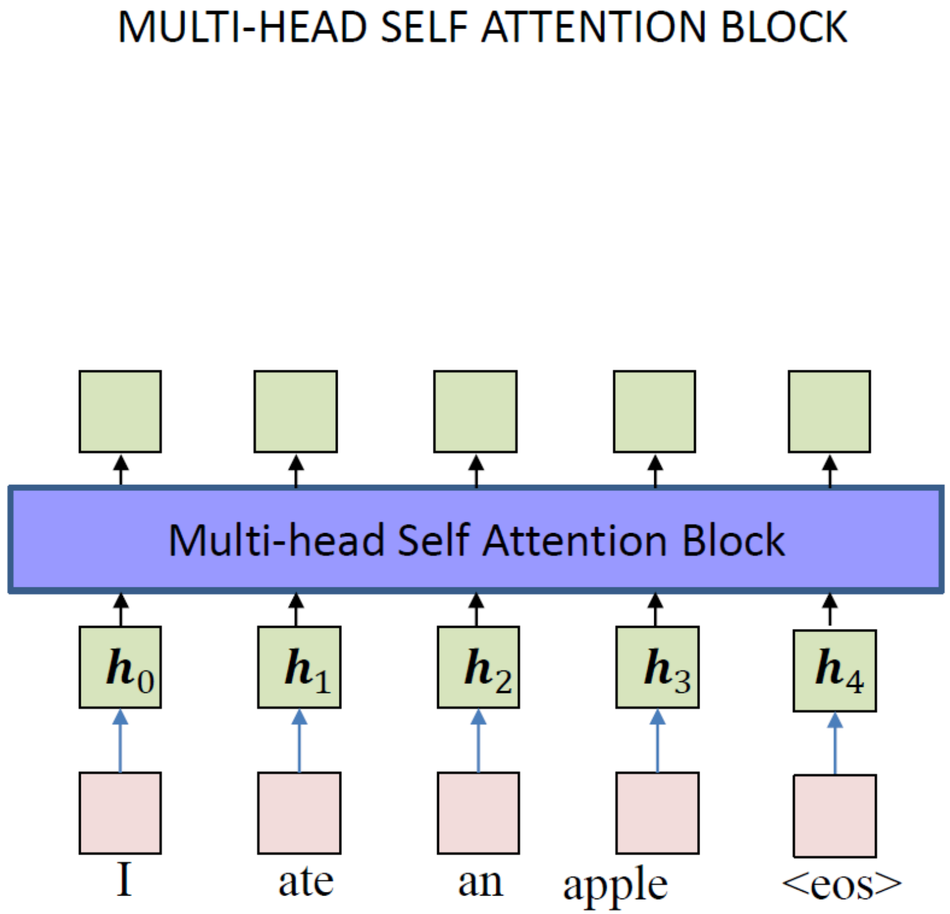
$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \ldots; o_i^H]$$

$$y_i = MLP(o_i)$$

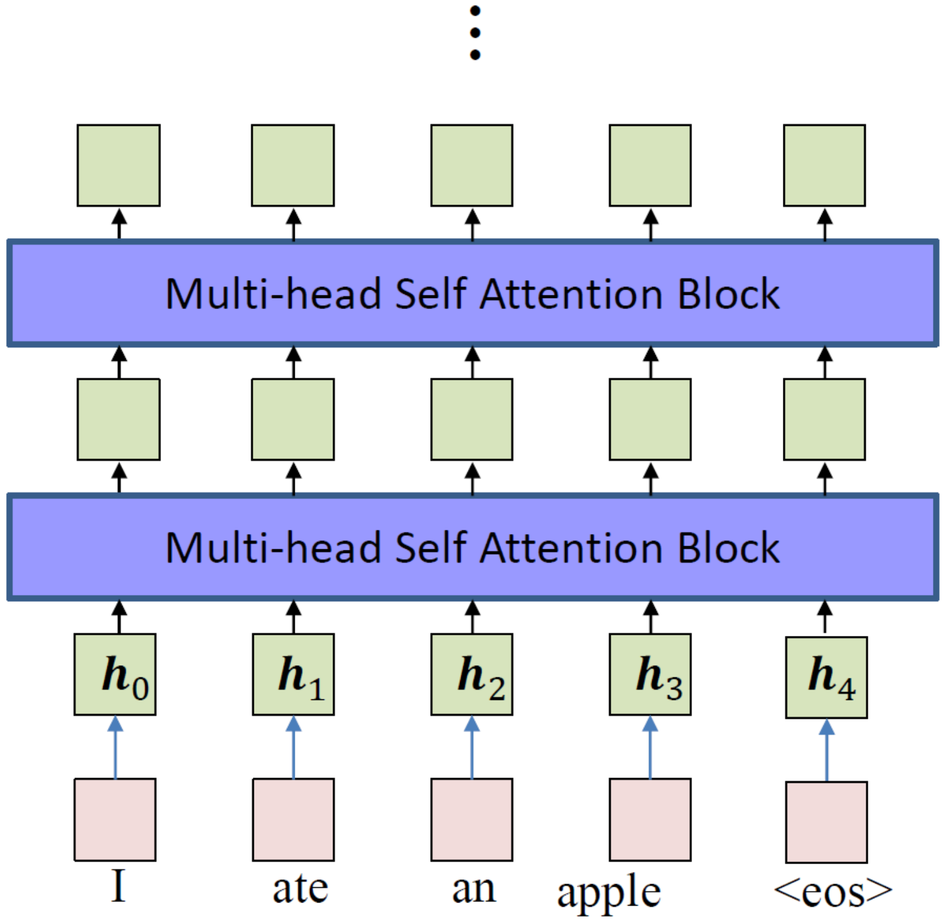


Multi-Head Self-Attention

- The *encoder* can include many layers of such blocks.
- No need for *recurrence*.

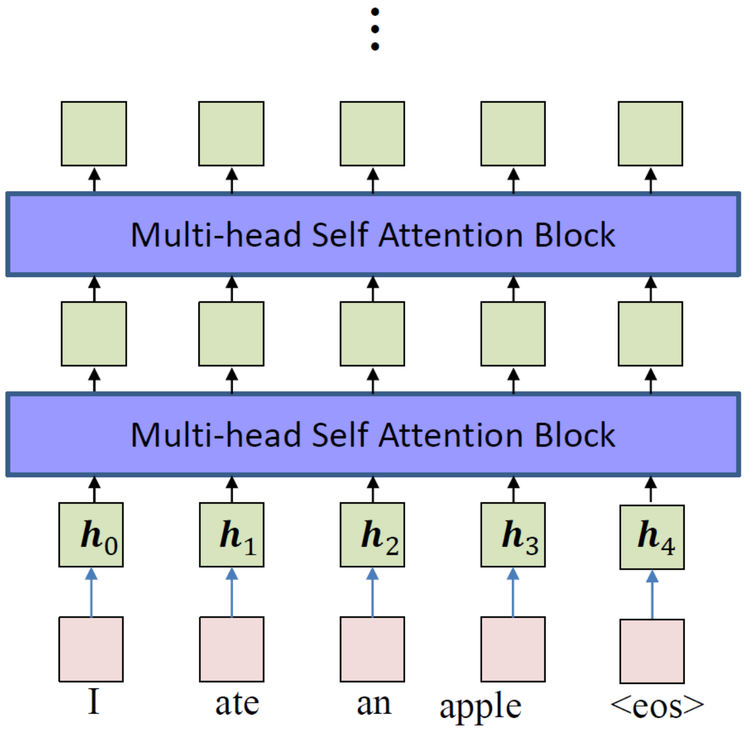
We transformed RNNs into standard deep neural networks!

Standard deep neural networks have a vertical structure, unlike the horizontal structure of RNNs.



Multi-Head Self-Attention

- The encoder in a sequence-to-sequence model can replace recurrence via a series of "multi-head self attention" blocks.
- But attention framework ignores *relative position*.
  - This framework does not consider position and distance.

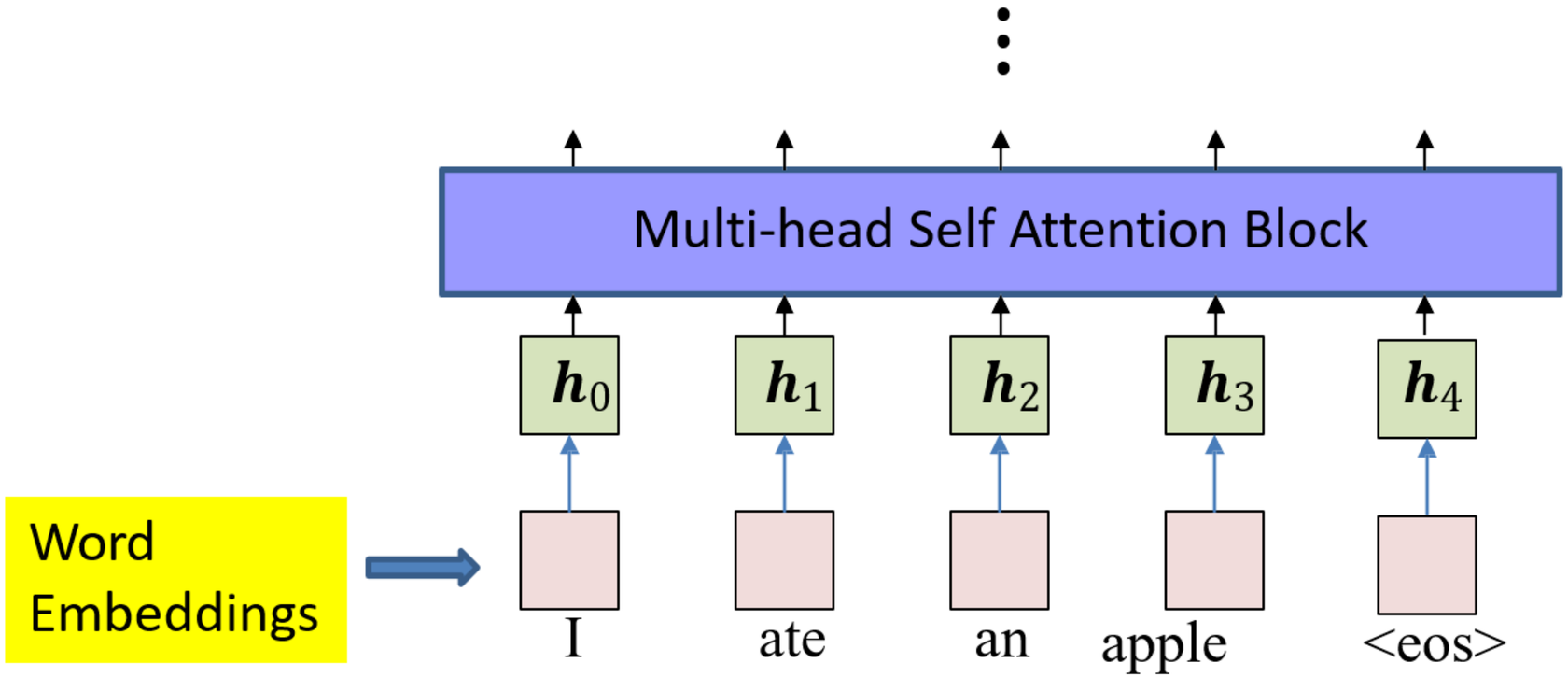


Outline

- Recap
- Self-Attention
- **Position Embeddings**
- Masked Self-Attention
- Transformer
- Practice & Discussion

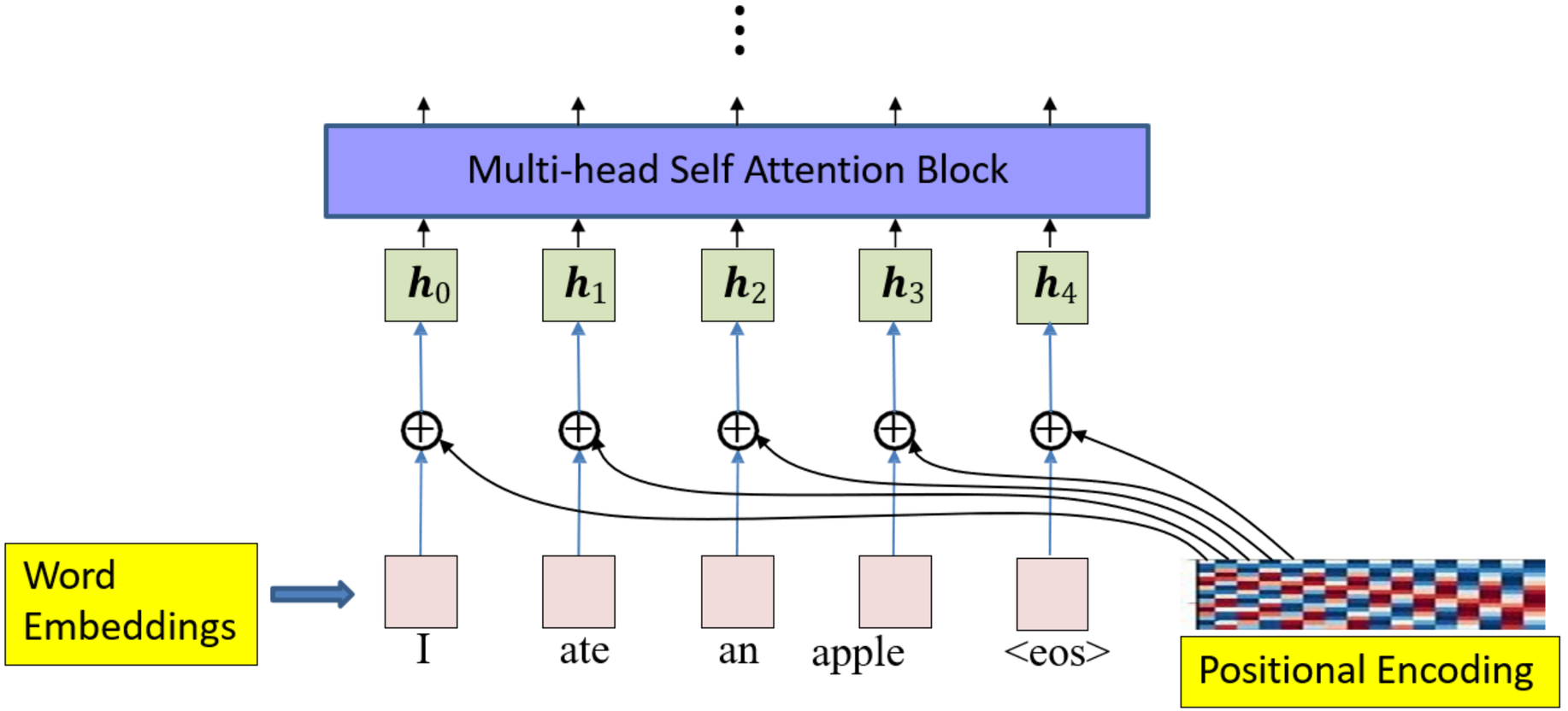
Position Embeddings

- Note that the inputs are actually word embeddings.



Position Embeddings

- Note that the inputs are actually word embeddings.
- We add a "positional" encoding to them to capture the relative distance from one another.

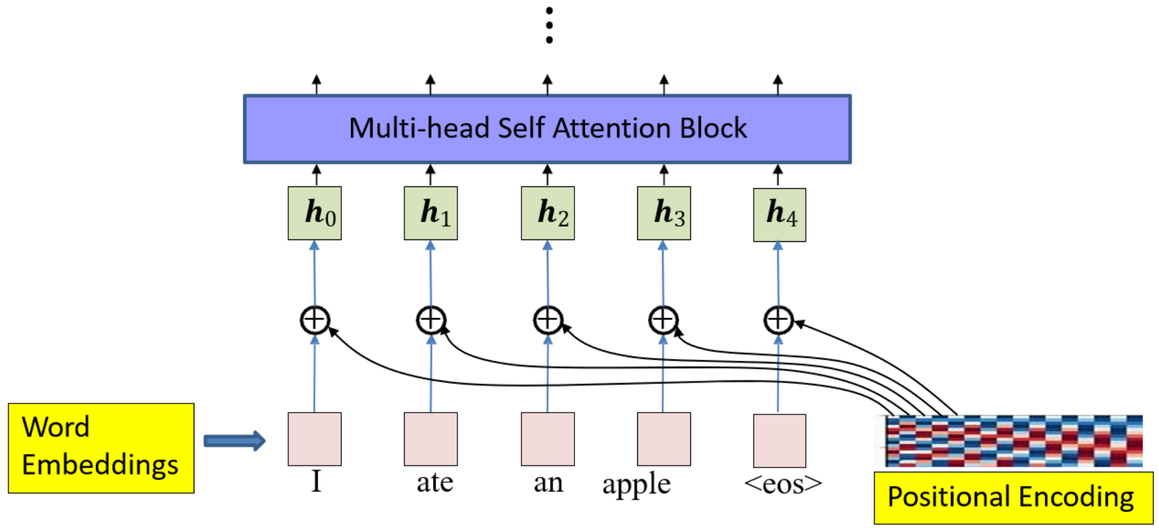


Position Embeddings

- Positional Encoding: A sequence of vectors $P_0, \cdots, P_N$ to encode position
  - Every vector is unique.
  - Relationship between P_t and P_{t+c} only depends on the distance between them

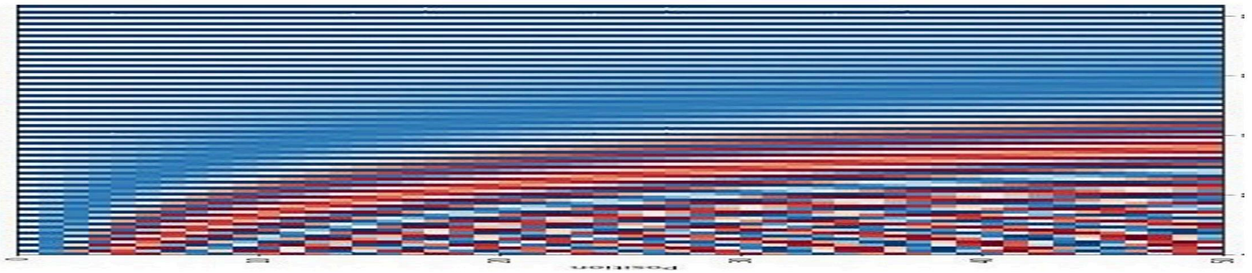
$$P_{t+c} = M_c P_t$$

- The linear relationship between P_t and P_{t+c} enables the net to learn shift-invariant (平移不变性) "gap" dependent relationships.



Position Embeddings: Sinusoidal

- A vector of sines and cosines of a harmonic series of frequencies (谐波频率)
  - Every $2l$-th component of P_t is $\sin w_l t$.
  - Every $(2l+1)$-th component of P_t is $\cos w_l t$.
- Never repeats.
- Has the *linearity* property required.



Here, d is the dimensionality of the vector, and $d = 2 \times l$.

$$P_t = \begin{bmatrix} \sin \omega_1 t \\ \cos \omega_1 t \\ \sin \omega_2 t \\ \cos \omega_2 t \\ \vdots \\ \sin \omega_{d/2} t \\ \sin \omega_{d/2} t \end{bmatrix}$$

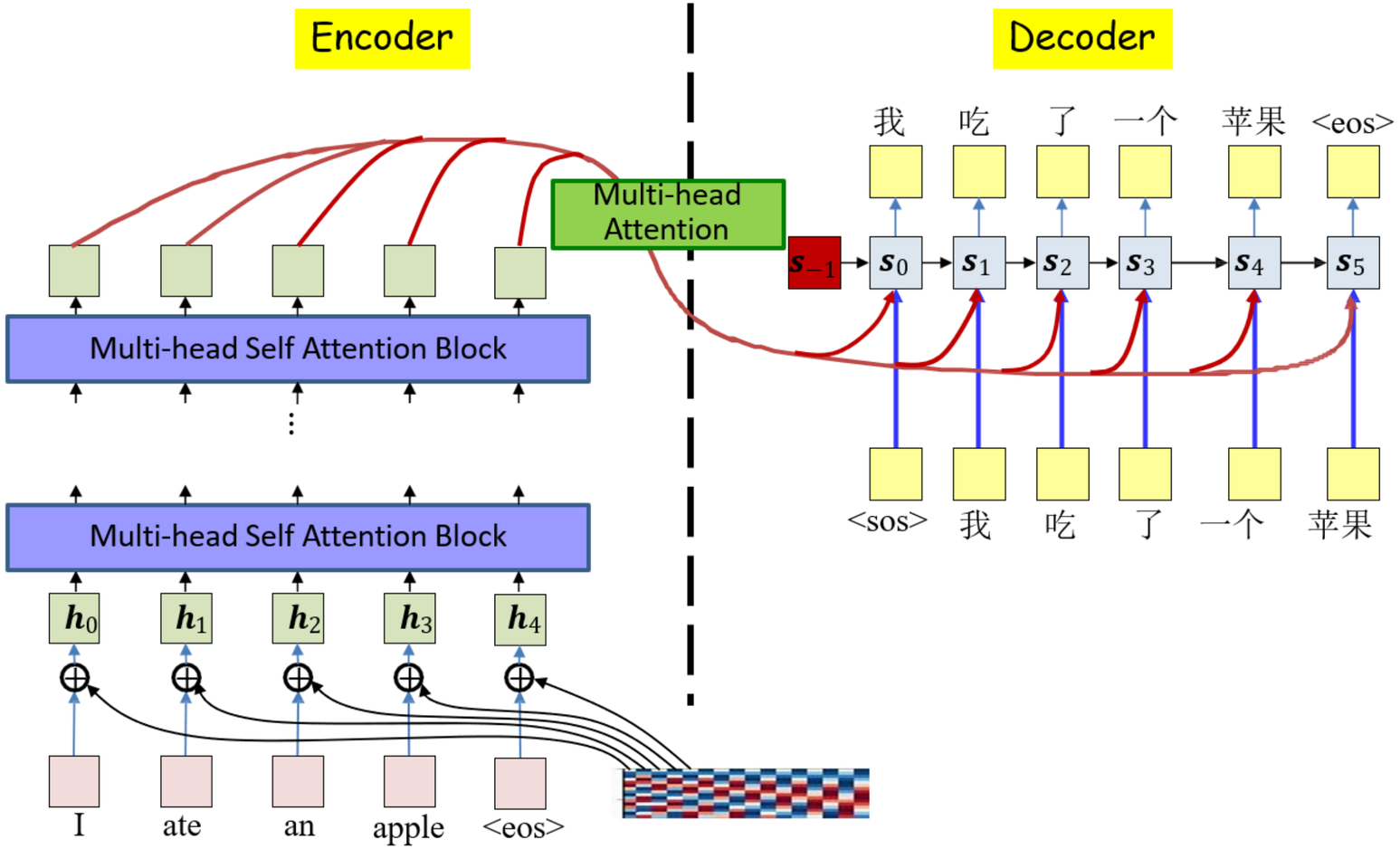
$$\omega_l = \frac{1}{10000^{2l/d}}$$

$$P_{t+c} = M_c P_t$$

$$M_c = diag\left(\begin{bmatrix} \cos \omega_l c & \sin \omega_l c \\ -\sin \omega_l c & \cos \omega_l c \end{bmatrix}, l = 1 \ldots d/2\right)$$

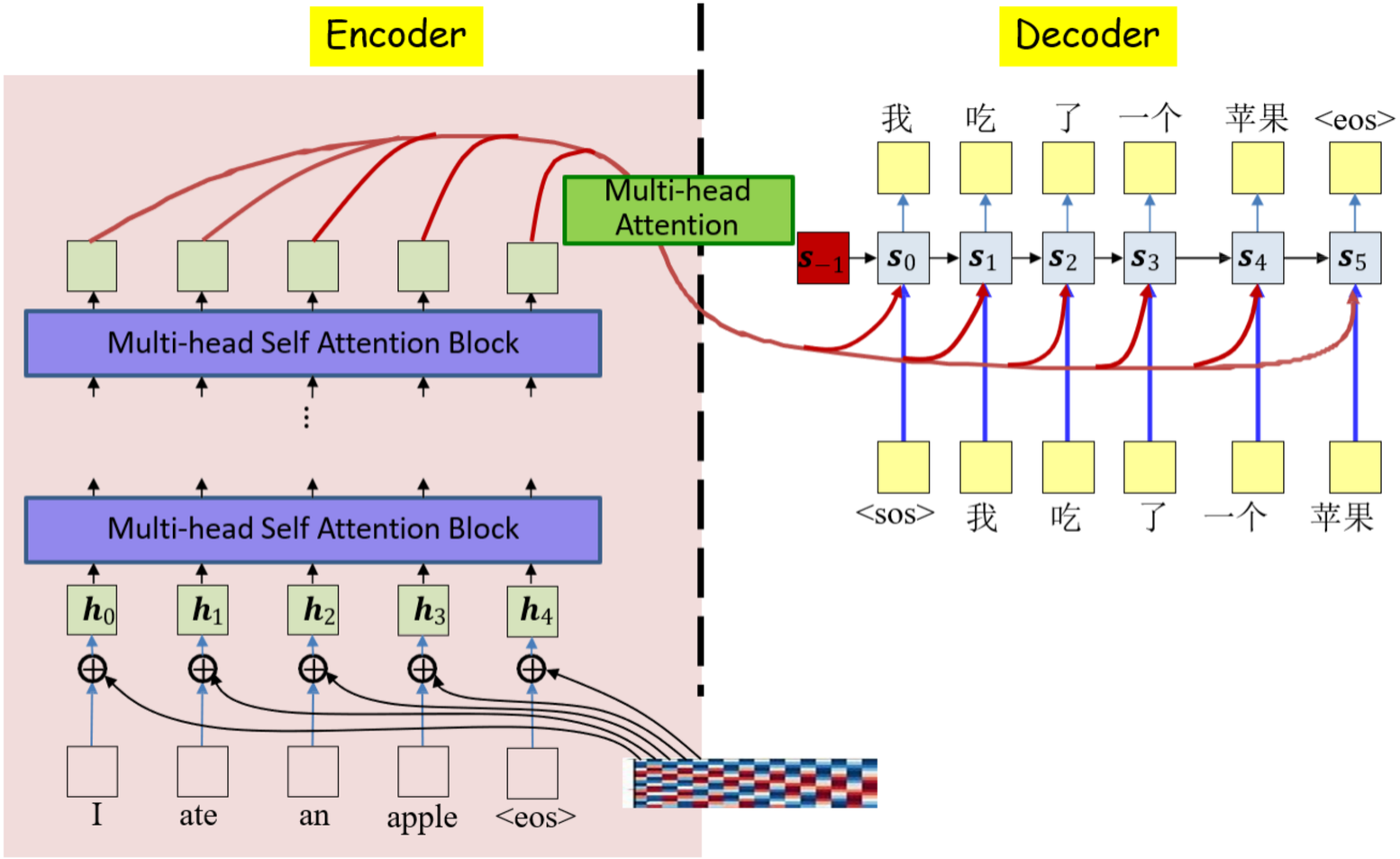
Position Embeddings

- The linear relationship between P_t and P_{t+c} enables the net to learn shift-invariant "gap" dependent relationships.



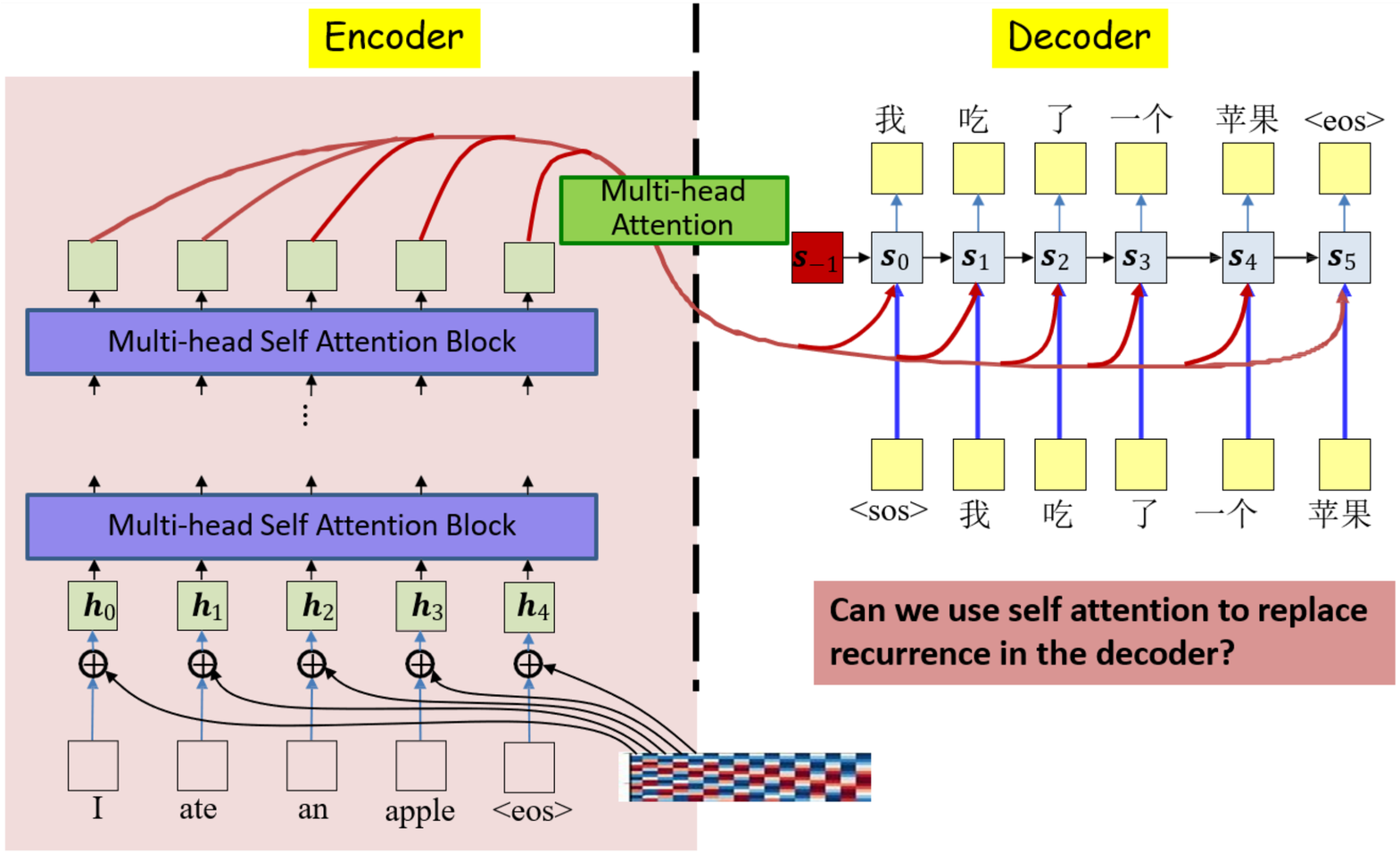
Position Embeddings

- The self-attending encoder!!



Position Embeddings

- The self-attending encoder!!

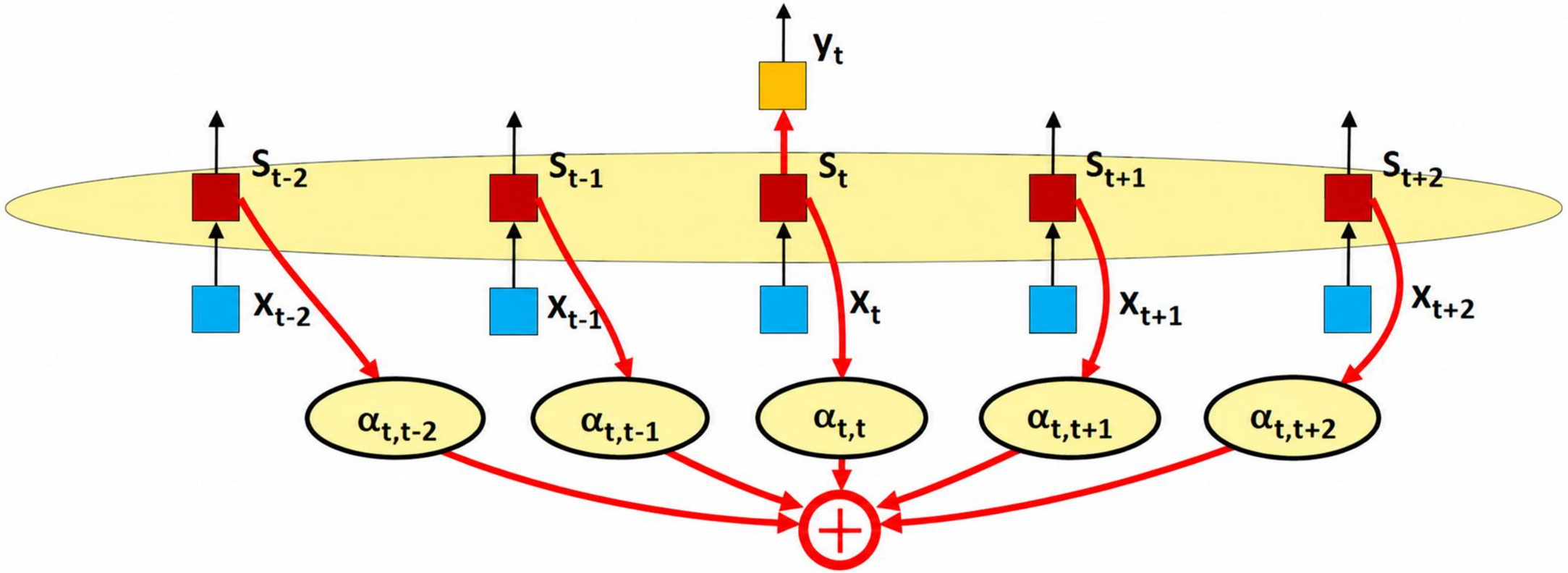


Outline

- Recap
- Self-Attention
- Position Embeddings
- **Masked Self-Attention**
- Transformer
- Practice & Discussion

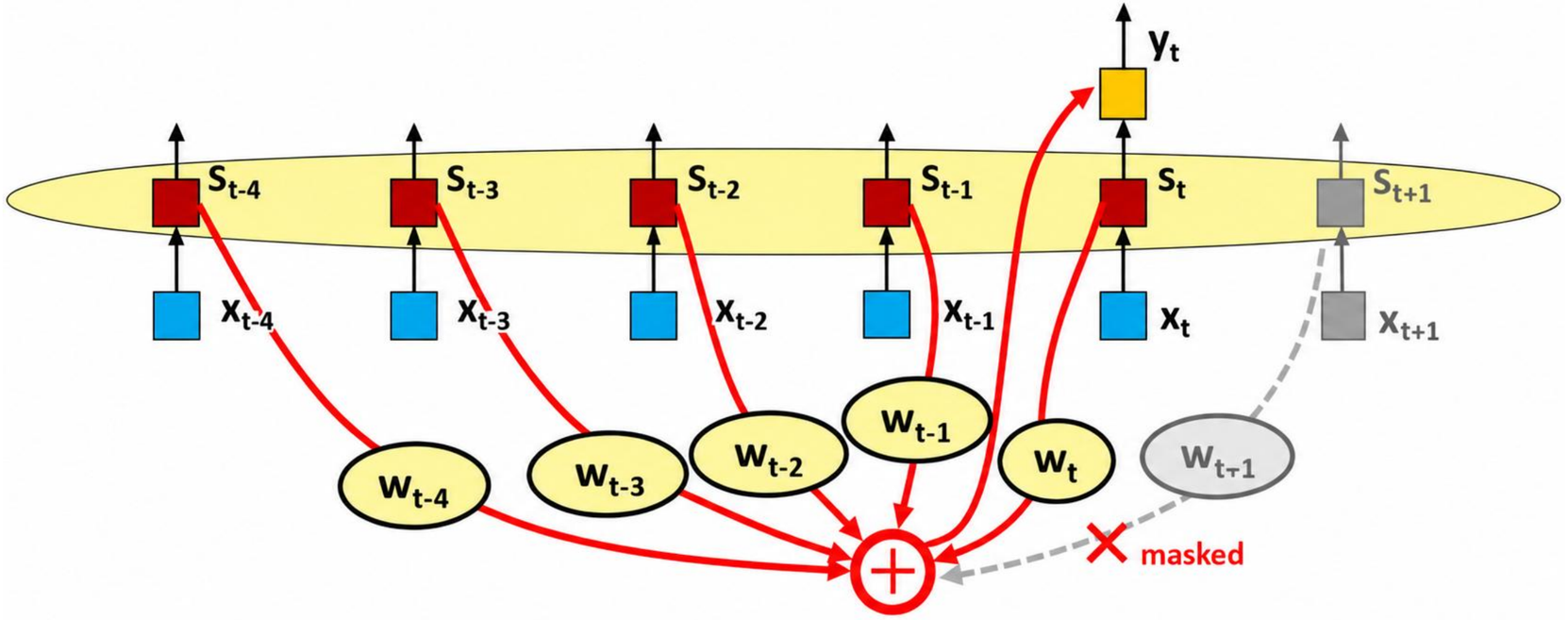
Masked Self-Attention

- Self-attention in encoder: Can use input embedding at time $t+1$ and further to compute output at time t, because all inputs are available.



Masked Self-Attention

- **Self-attention in decoder: Decoder is sequential.**
  - Each word is produced using the previous word as input.
  - Only embeddings until time t are available to compute the output at time t.
- The attention will have to be "masked", forcing attention weights for $t+1$ and later to **0**.



Masked Self-Attention

- Mathematically, self-attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- For masked attention, we apply a mask M before *softmax*:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- Where M is:
  - 0 for allowed positions.
  - $-\infty$ for masked positions (future tokens).

Masked Self-Attention Block

- The "masked self attention block" includes an *MLP* after the masked self attention.
  - Like in the encoder.

$q_i = \boldsymbol{W}_q h_i$

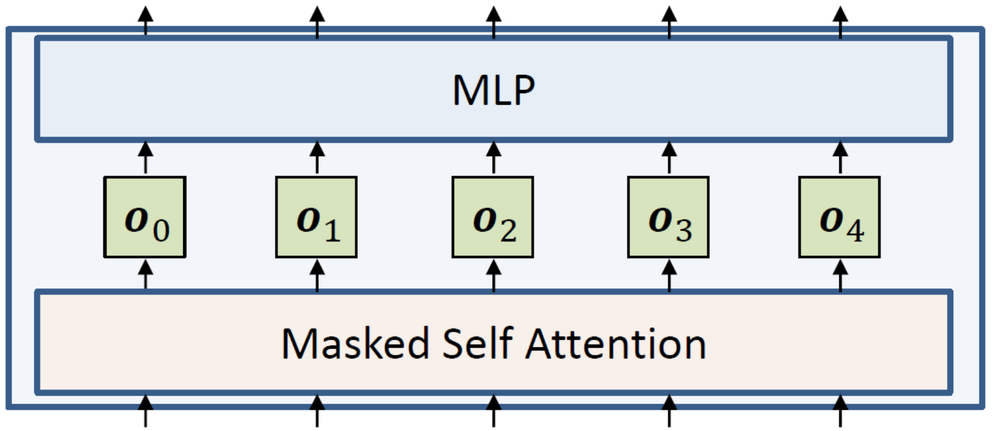
$k_i = \boldsymbol{W}_k h_i$

$v_i = \boldsymbol{W}_v h_i$

$e_{ij} = q_i^T k_j$

$w_{i0}, \ldots, w_{ii} = softmax(e_{i0}, \ldots, e_{ii})$

$o_i = \sum_{j=0}^{i-1} w_{ij} v_j$



Masked Self-Attention Block

- The "masked self attention block" includes *multiple masked attention heads*.
  - Like in the encoder.

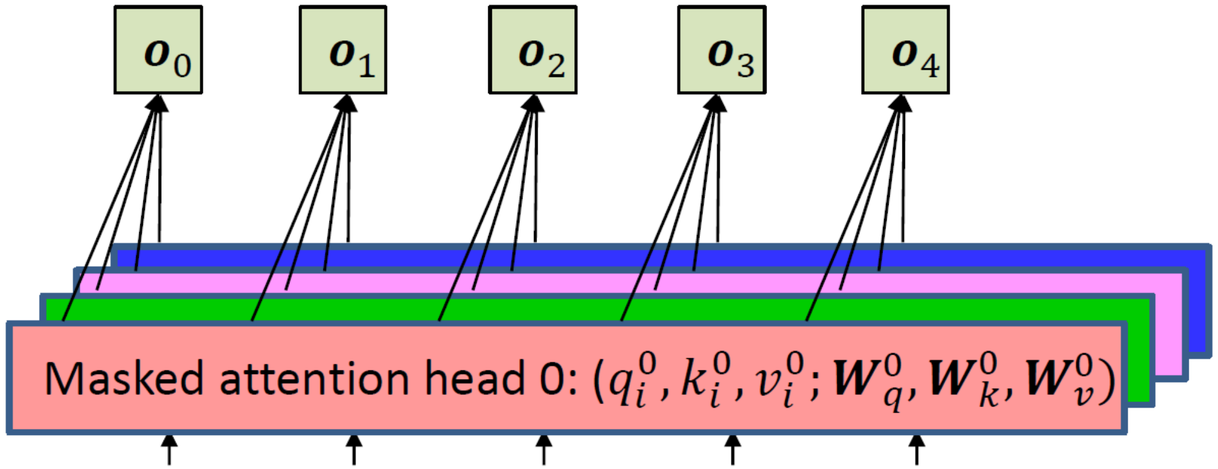
$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:i-1}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$



Masked Self-Attention Block

- We have the "masked multi-head self attention block".
  - Like in the encoder.

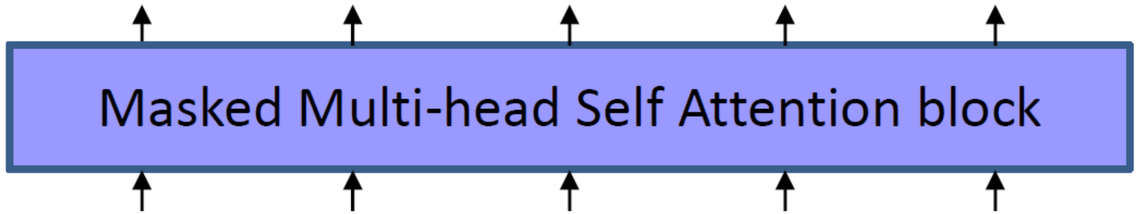
$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

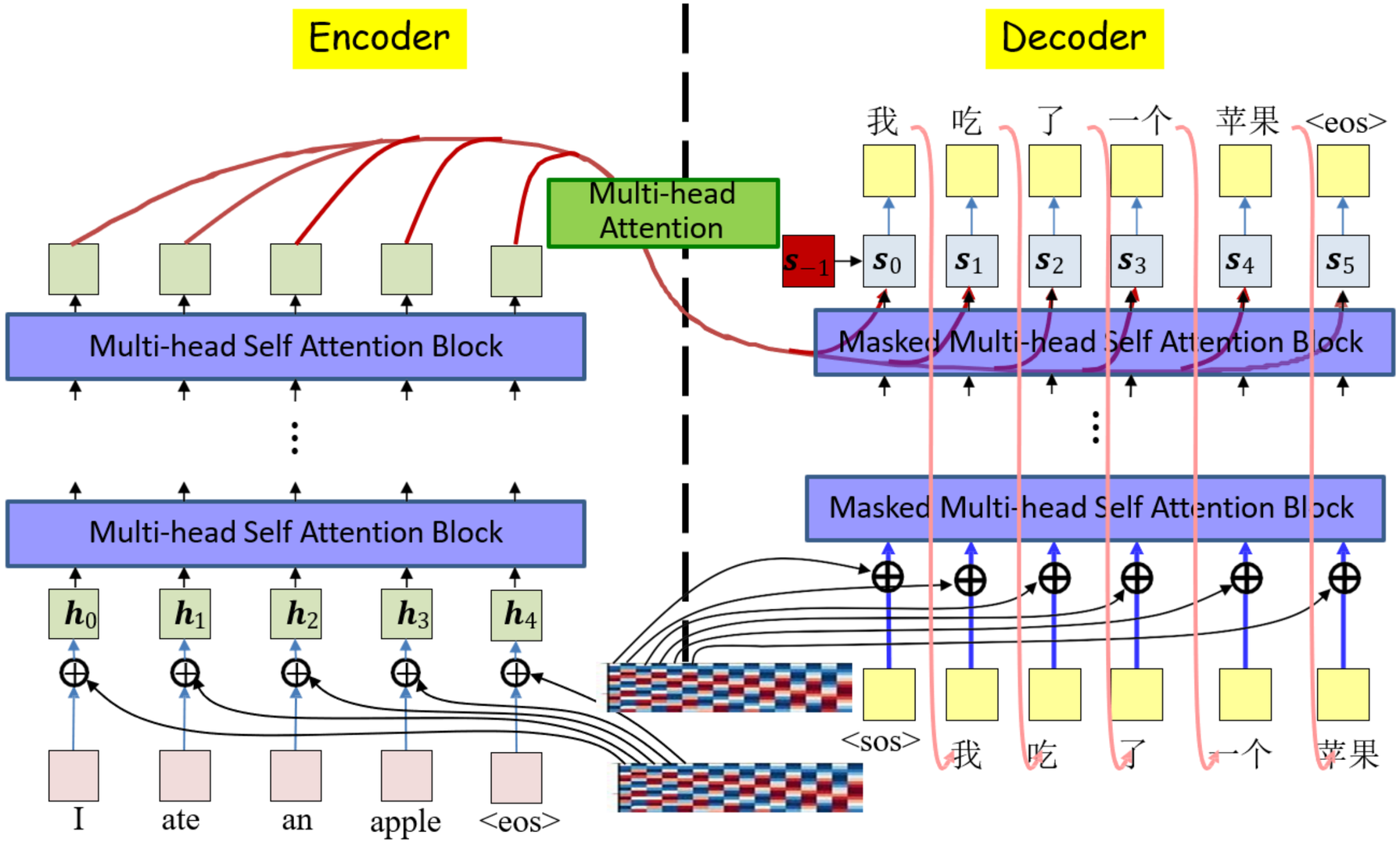
$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:i-1}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$



Masked Self-Attention Block

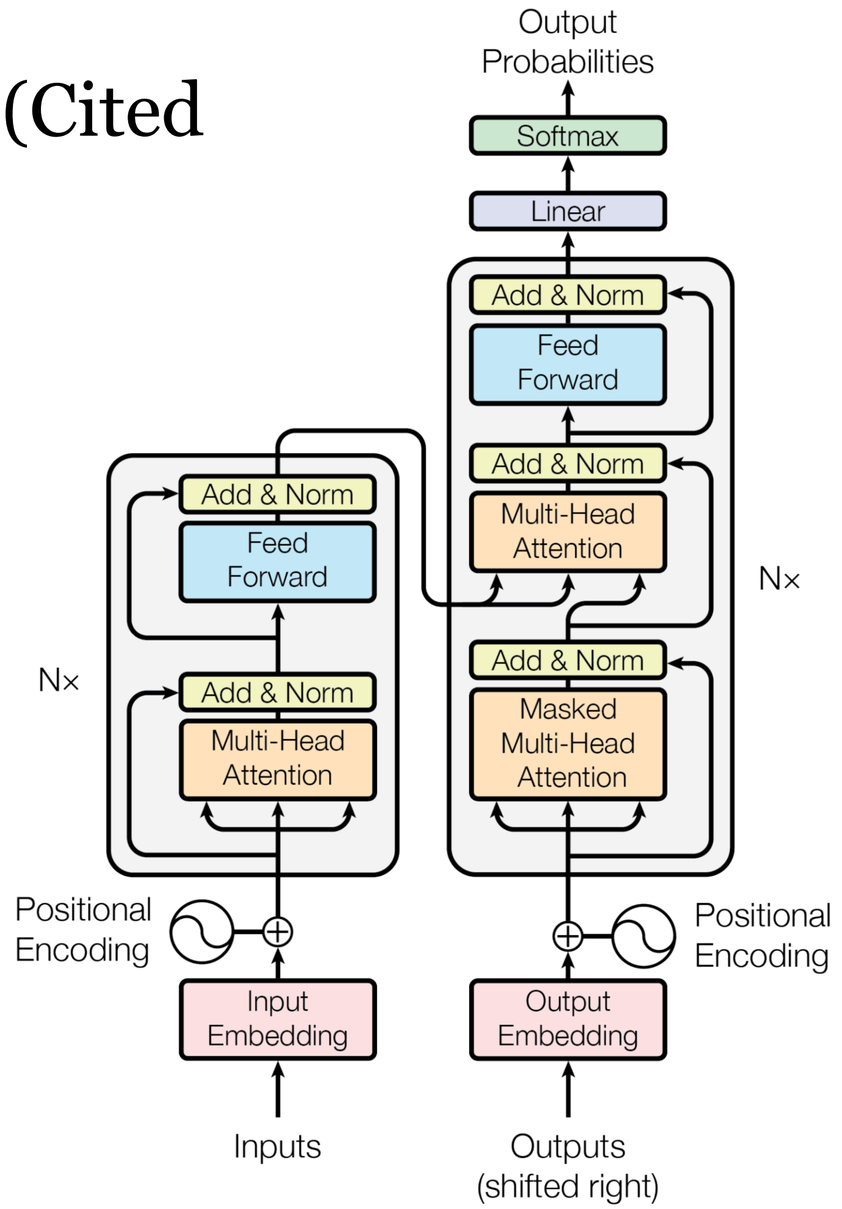


Outline

- Recap
- Self-Attention
- Position Embeddings
- Masked Self-Attention
- **Transformer**
- Practice & Discussion

Transformer: Attention is All You Need

- **Transformer**: Attention is all you need (Cited by 275,368, 2026/10/12).
  - Utilize self-attention
  - Simple attention scoring function
  - Added linear layers for Q, K and V
  - Multi-head attention
  - Added positional encoding
  - Added residual connection
  - Simulate decoding by masking



Transformer: Scaled Dot-Product Attention

- Transformer employs *scaled dot-product attention*

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

where d_k is the dimension of *keys* and *queries*.

> "*We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$*"

Transformer: Batch Normalization

- **Definition**: In each layer of the neural network, normalize each feature of every mini-batch to keep its output distribution stable (with a mean of 0 and a variance of 1). This technique improves training speed and stability.
- Suppose the output of an intermediate layer in the neural network is a matrix $x \in \mathbb{R}^{B \times D}$.
  - Here, B represents the batch size, and D denotes the feature dimension (e.g., the number of neurons in the hidden layer). For each feature dimension j (i.e., each column), perform the following operations:

Transformer: Batch Normalization

- Step 1: Calculate the mean and variance within the mini-batch
  - Mean: $\mu_j = \frac{1}{B}\sum_{i=1}^{B} x_{i,j}$ Variance: $\sigma_j^2 = \frac{1}{B}\sum_{i=1}^{B}(x_{i,j}-\mu_j)^2$
- Step 2: Normalization
  - $\hat{x}_{i,j} = \frac{x_{i,j}-\mu_j}{\sqrt{\sigma_j^2+\epsilon}}$, where ϵ is a small constant to avoid division by zero.
- Step 3: Scaling and Shifting (learnable parameters)
  - $y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$
  - Here, γ_j and β_j are learnable parameters, which control scaling and shifting respectively (enabling *BN* to maintain expressive power).

Transformer: Layer Normalization

- Batch Normalization is helpful but hard to use with sequence models.
  - Sequences are different lengths, makes normalizing across the batch hard.
  - Sequences can be very long, so we sometimes have small batches.
- Layer Normalization: like batch norm, but not across the batch.
- **Idea**: Normalize all features (along the vector dimension) within a single sample to achieve a mean of 0 and a standard deviation of 1.

Assume the input is a representation vector of a sample:

- Input: $x = [x_1, x_2, \ldots x_d] \in \mathbb{R}^d$,

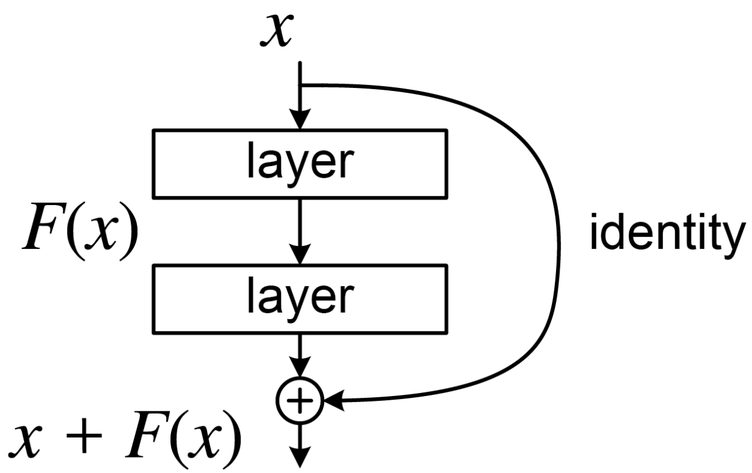
  where d is the dimension of the hidden state (e.g., 512).

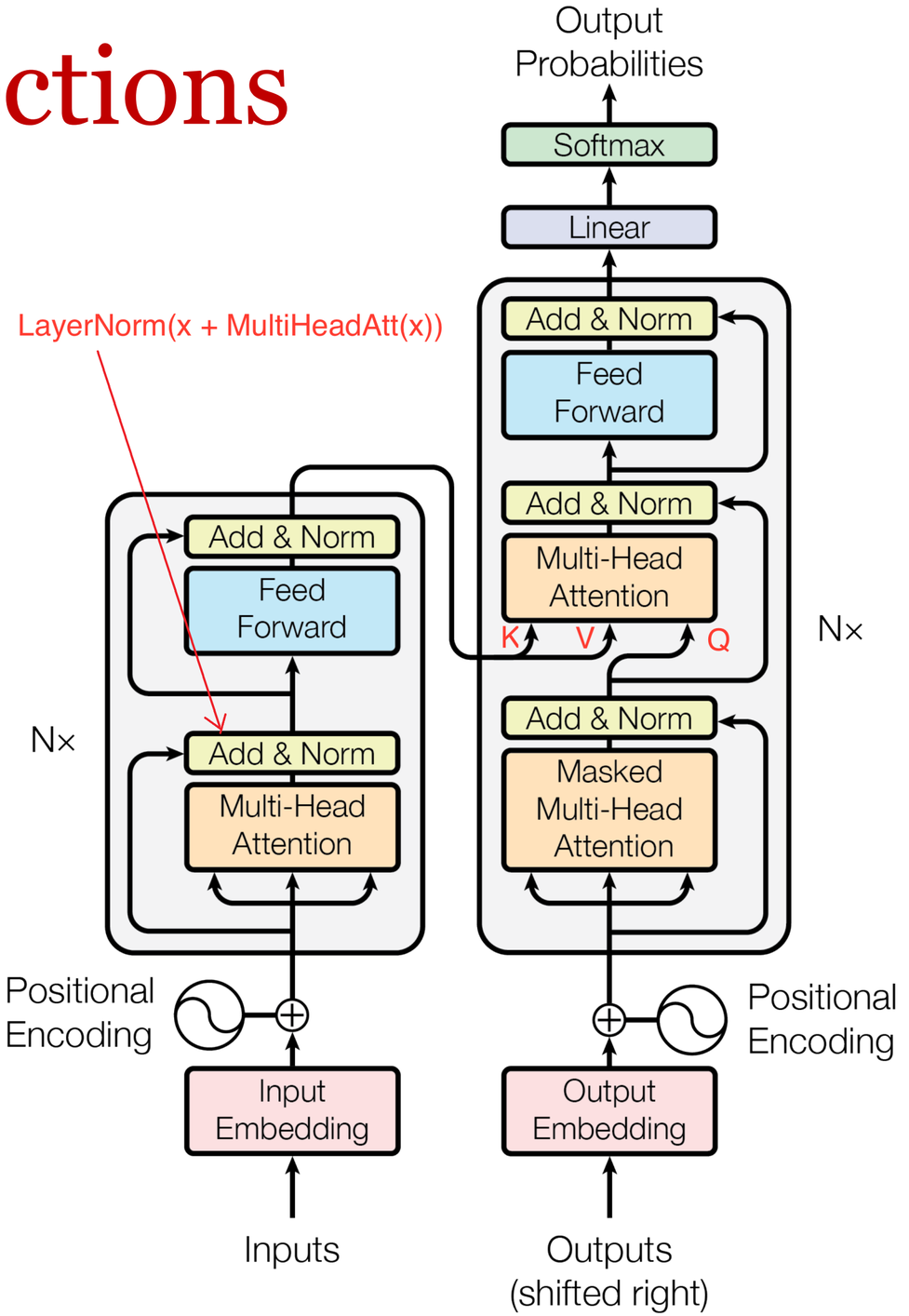
Transformer: Layer Normalization

- Step 1: Calculate the mean and variance across the feature dimension
  - Mean: $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$
  - Variance: $\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$
- Step 2: Normalization
  - $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
  - where ϵ is a small constant to avoid division by zero.
- Step 3: Scaling and Shifting (Learnable Parameters)
  - $y_i = \gamma \hat{x}_i + \beta$
  - Here, γ and β are learnable parameters to control scaling and shifting.

Transformer: Residual Connections

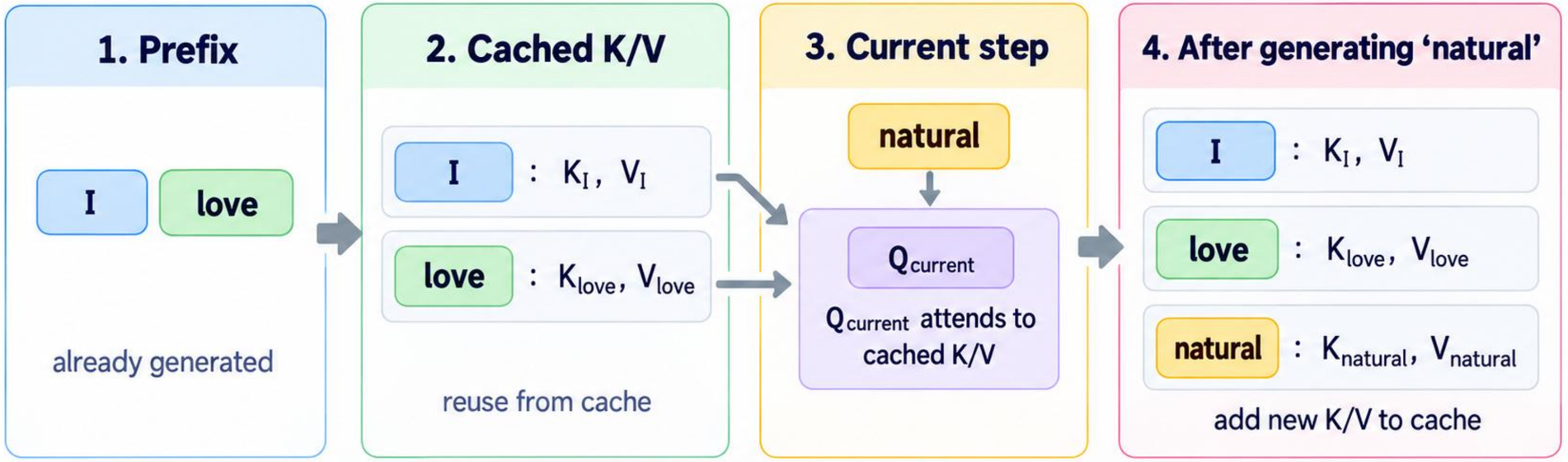
- Residual connections make the output of each layer equal to the *sub-layer output plus the original input*.
  - Prevent information loss and gradient vanishing.
  - Enhance the stability of model training.



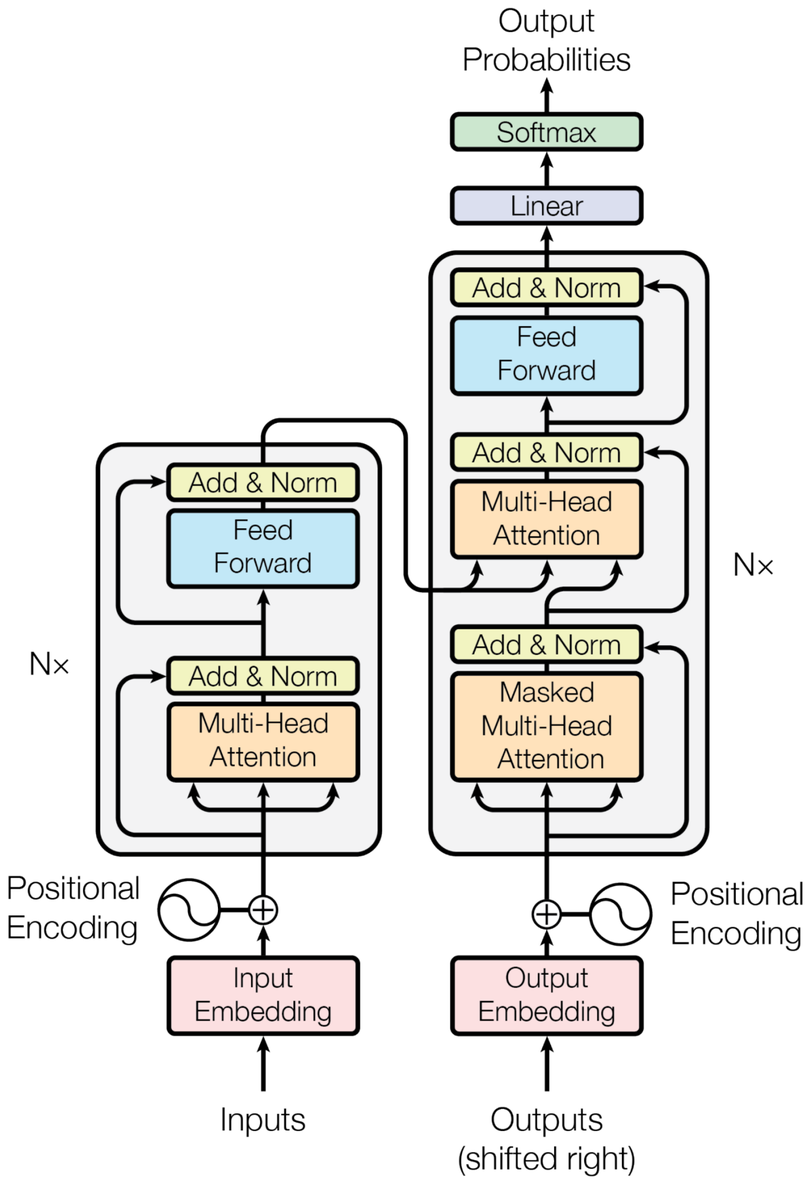


Transformer: KV Caching

- In *causal decoding*, attention for previous tokens is recomputed at each step, although only the new token's attention is needed.
- KV cache: cache the previous *Keys* and *Values*, and only calculate the attention for the new token.



Transformer: pseudocode



High-Level Transformer

```
def transformer(src_ids, tgt_ids):
    # 1. 嵌入输入序列和目标序列（词ID → 向量）
    src_embed = embed(src_ids)  # [B, src_len, d_model]
    tgt_embed = embed(tgt_ids)  # [B, tgt_len, d_model]

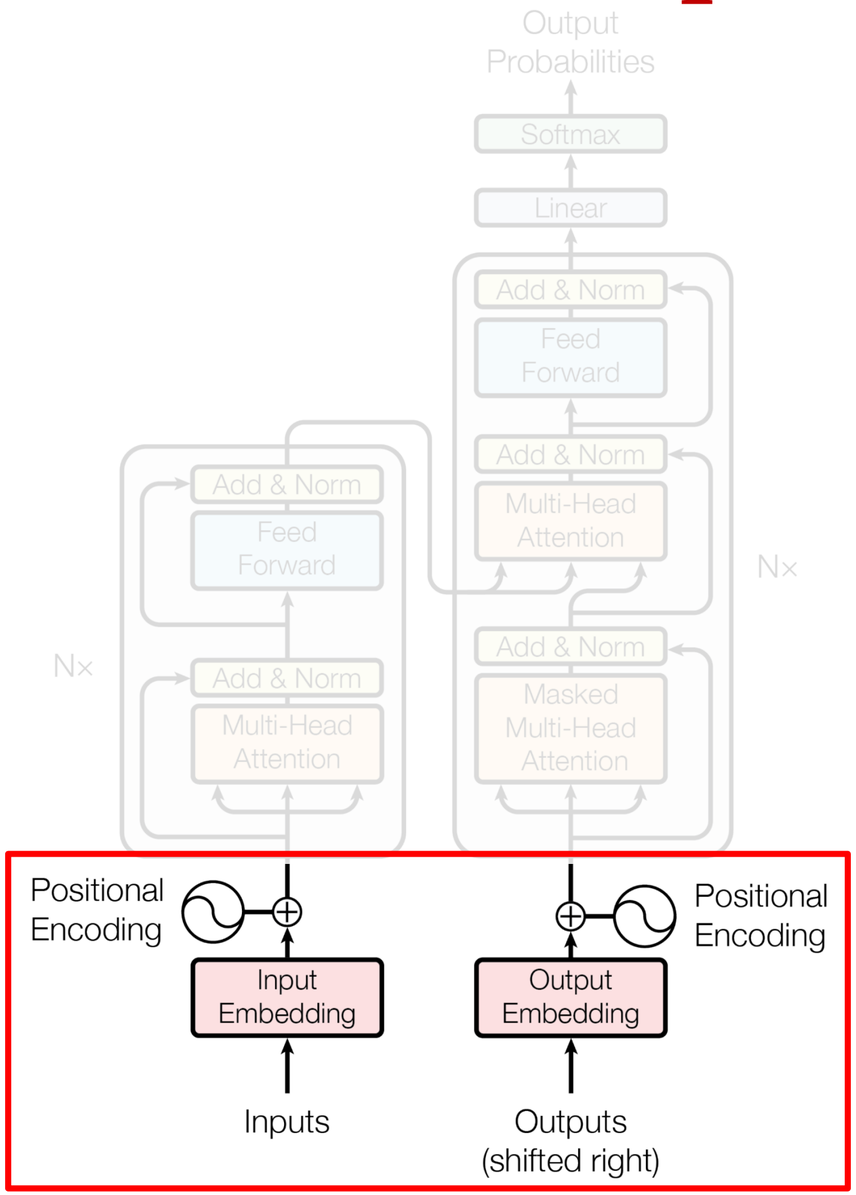
    # 2. 加上位置编码，保留词序信息
    src_encoded = add_positional_encoding(src_embed)
    tgt_encoded = add_positional_encoding(tgt_embed)

    # 3. 编码器处理源语言序列
    enc_output = encoder(src_encoded)  # [B, src_len, d_model]

    # 4. 解码器处理目标序列和编码器输出
    dec_output = decoder(tgt_encoded, enc_output)  # [B, tgt_len, d_model]

    # 5. 将输出向量映射为词表概率分布
    return project_to_vocab(dec_output)  # [B, tgt_len, vocab_size]
```

Transformer: pseudocode

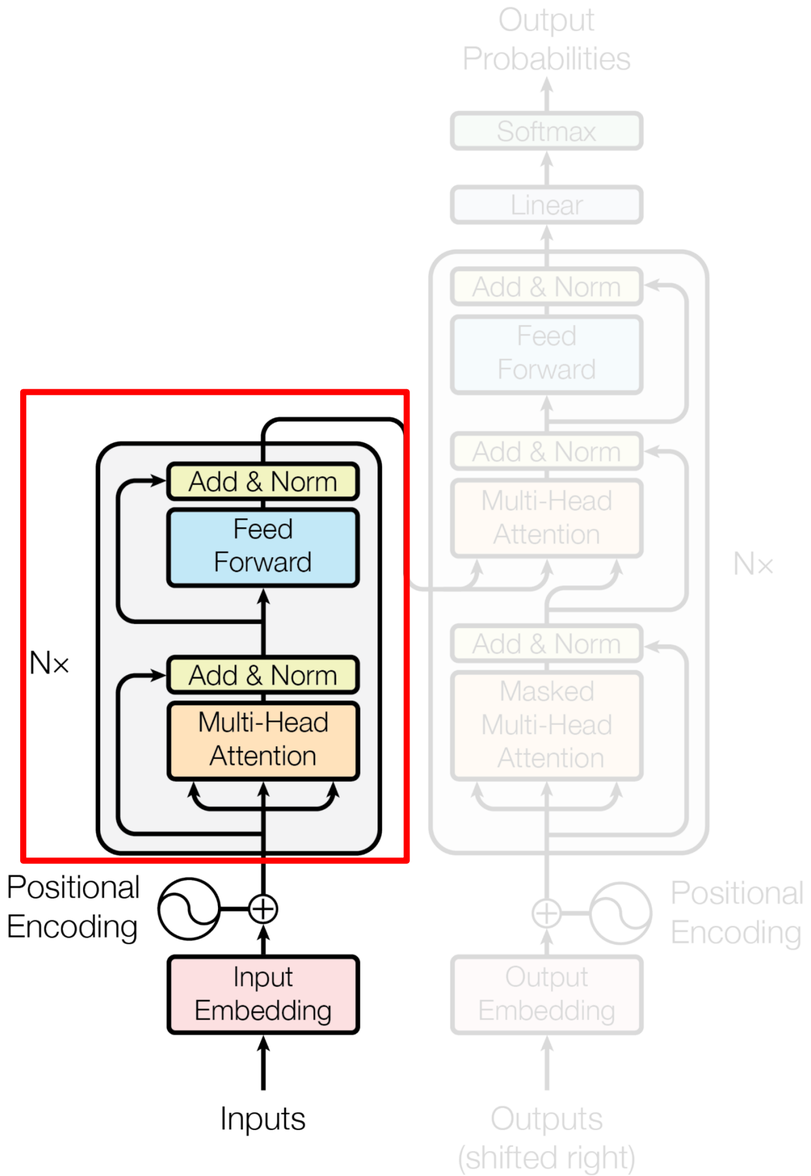


Word and Position Embeddings

```
def embed(token_ids):
    # 将每个词ID映射为 d_model 维向量
    return embedding_table[token_ids]

def add_positional_encoding(x):
    # 为每个时间步加上固定位置向量（正余弦函数）
    return x + positional_encoding[:len(x)]
```

Transformer: pseudocode



Encoder

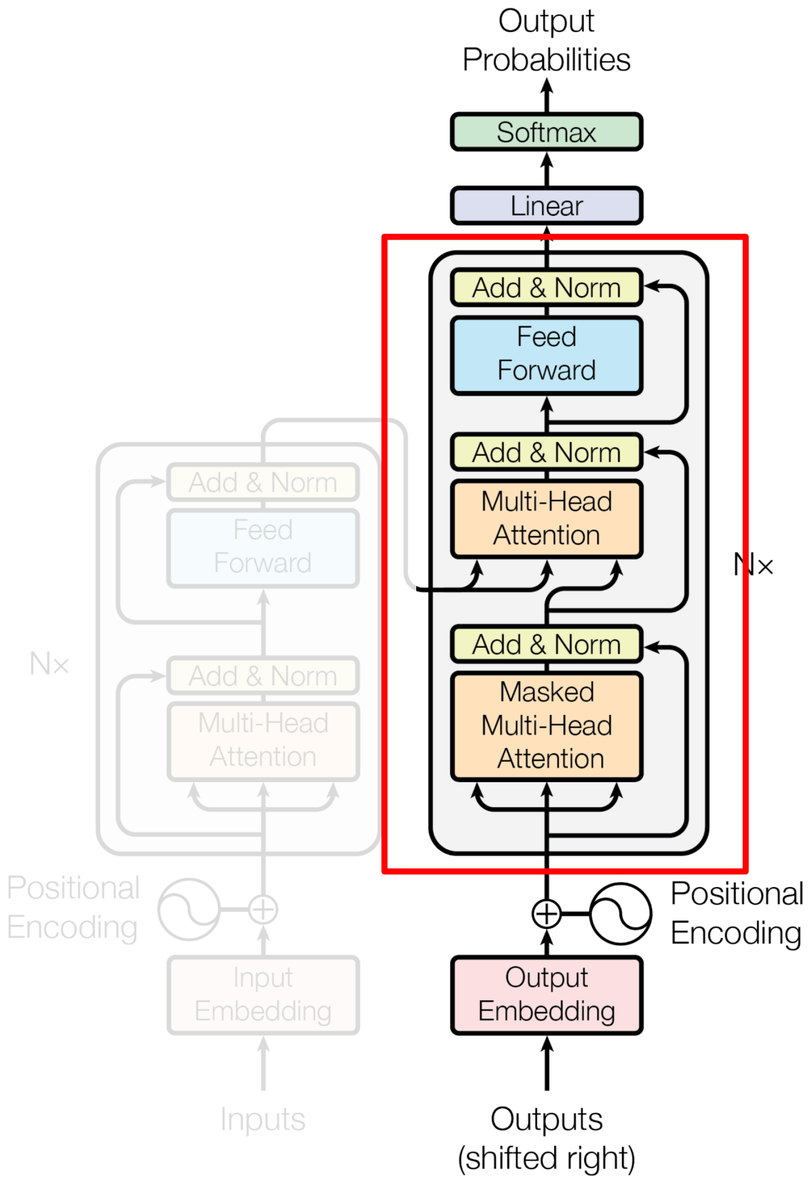
```
def encoder(x):
    for _ in range(N):  # 堆叠 N 层编码器
        x = encoder_layer(x)
    return x
```

```
def encoder_layer(x):
    # 1. 多头自注意力（每个 token 看所有其他 token）
    attn_out = multi_head_attention(x, x, x)  # Q=K=V=x

    # 2. 残差连接 + LayerNorm（让训练更稳定）
    x = layer_norm(x + attn_out)

    # 3. 前馈网络（增强非线性建模能力）
    ffn_out = feed_forward(x)
    x = layer_norm(x + ffn_out)
    return x
```

Transformer: pseudocode



Decoder

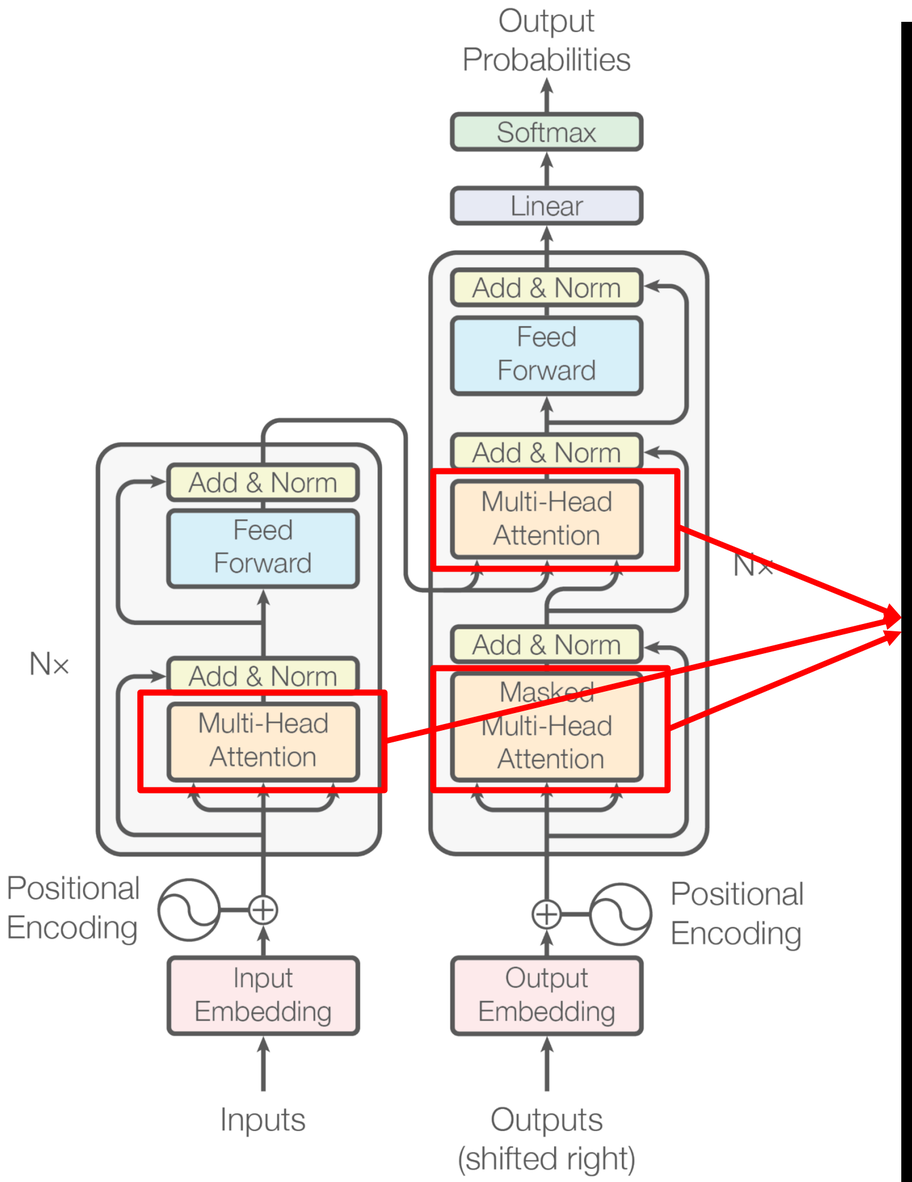
```
def decoder(x, enc_output):
    for _ in range(N):  # 堆叠 N 层解码器
        x = decoder_layer(x, enc_output)
    return x
```

```
def decoder_layer(x, enc_output):
    # 1. Masked 多头自注意力（只看当前位置及前面）
    self_attn = multi_head_attention(x, x, x, mask="future")
    x = layer_norm(x + self_attn)

    # 2. 编码器-解码器注意力（Query=解码，Key/Value=编码）
    cross_attn = multi_head_attention(x, enc_output, enc_output)
    x = layer_norm(x + cross_attn)

    # 3. 前馈网络
    ffn_out = feed_forward(x) //point wise
    x = layer_norm(x + ffn_out) //point wise
    return x
```

Transformer: pseudocode



Multi-head Attention

```
def multi_head_attention(q, k, v, mask=None):
    heads = []

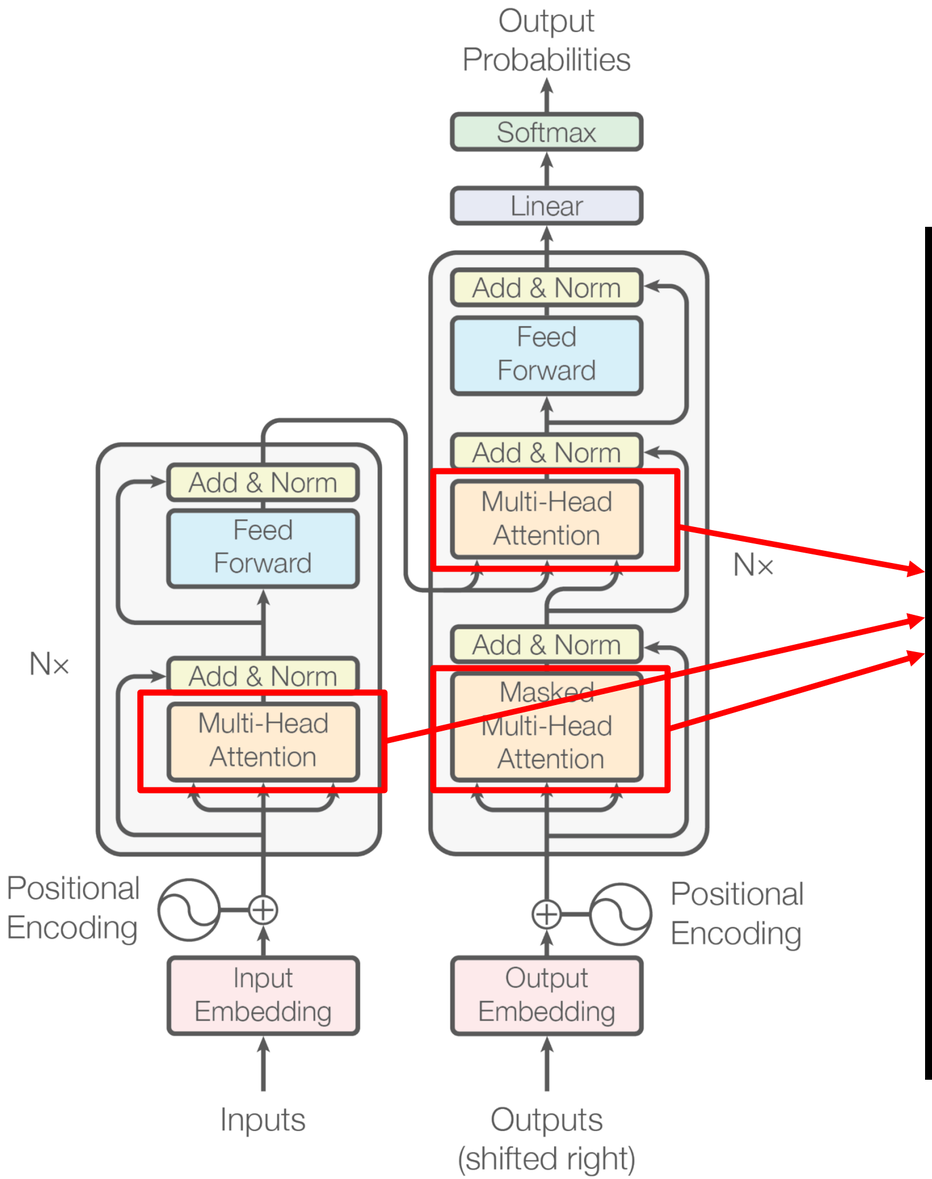
    for i in range(num_heads):
        # 每个头独立线性映射（低维：d_k = d_model / num_heads）
        q_i = linear_q[i](q)  # [B, T, d_k]
        k_i = linear_k[i](k)
        v_i = linear_v[i](v)

        # 每个头单独做 scaled dot-product attention
        head = scaled_dot_product_attention(q_i, k_i, v_i, mask)
        heads.append(head)

    # 拼接所有头的结果 → 合并成一个向量
    concat_output = concat(heads)  # [B, T, d_model]

    # 通过一个线性层融合所有头信息
    return final_linear(concat_output)
```

Transformer: pseudocode



Attention（Scaled Dot-Product Attention）

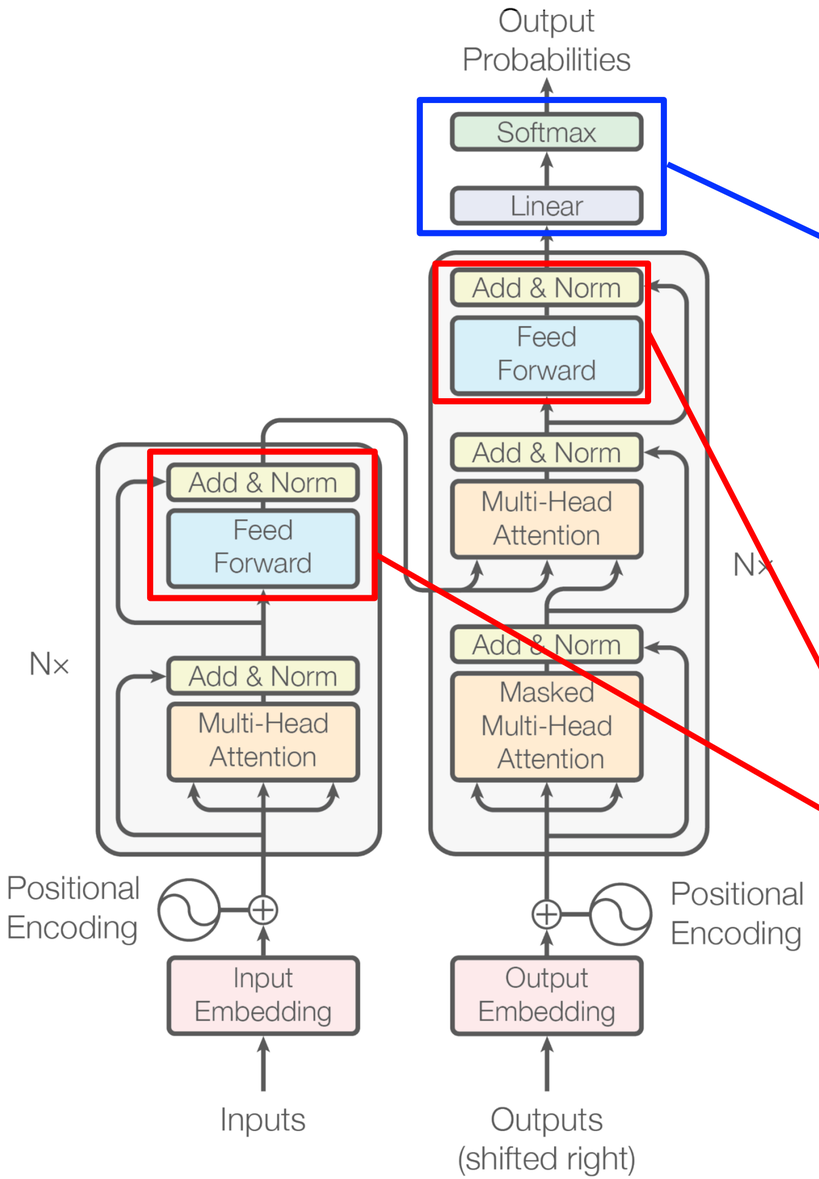
```
def scaled_dot_product_attention(q, k, v, mask=None):
    # 1. 计算注意力分数：QK^T / sqrt(d_k)
    scores = (q @ k.transpose) / sqrt(d_k)

    # 2. 应用 mask（如：防止看到未来）
    if mask:
        scores = scores.masked_fill(mask == 0, -inf)

    # 3. 归一化为权重
    weights = softmax(scores)

    # 4. 权重加权 Value 向量
    return weights @ v
```

Transformer: pseudocode



Output

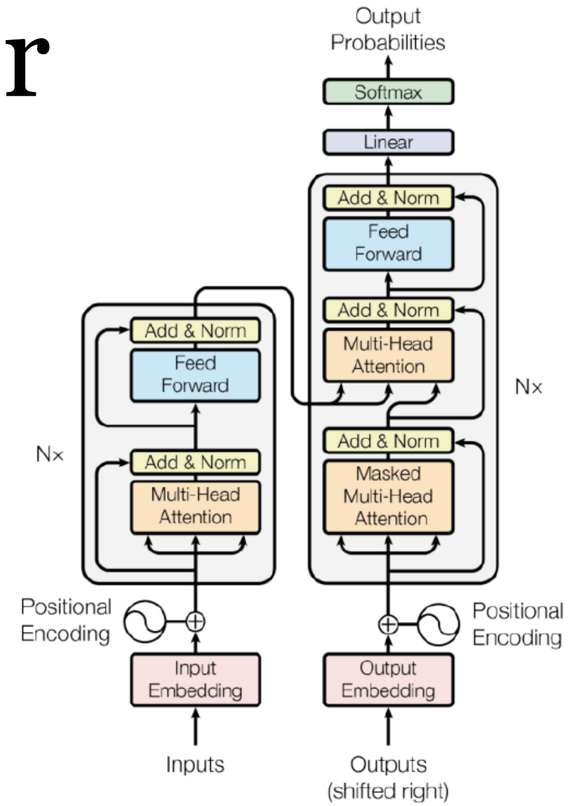
```
def project_to_vocab(x):
    # 将每个 token 的表示映射为词表上的概率
    return softmax(linear(x))  # [B, T, vocab_size]
```

FFN and LayerNorm

```
def feed_forward(x):
    # 两层线性网络，中间加 ReLU
    return linear2(relu(linear1(x)))

def layer_norm(x):
    # 对每个 token 的向量标准化（均值为0，方差为1）
    return normalize(x)
```

Transformer



From "Attention is all you need"

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Why so good?

Why so fast?

- Transformer: tremendous decrease in model computation for similar performance as state-of-art translation models.
- The last row in the table shows transformer performance.
- The final two columns show computational cost.

Transformer: Attention Example

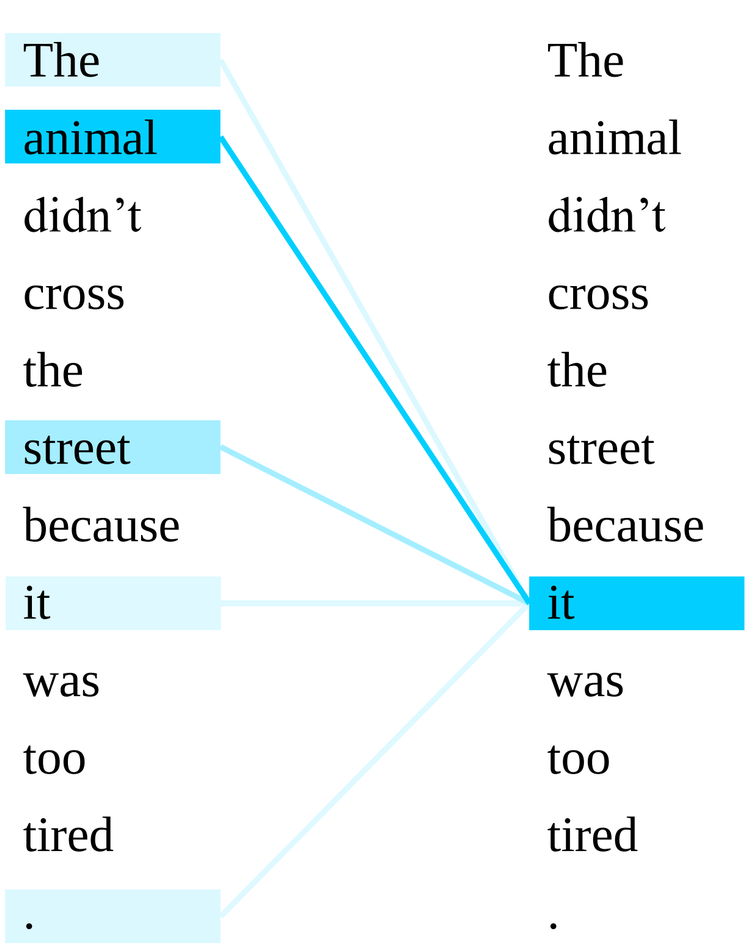
- What is the difference between the two examples?

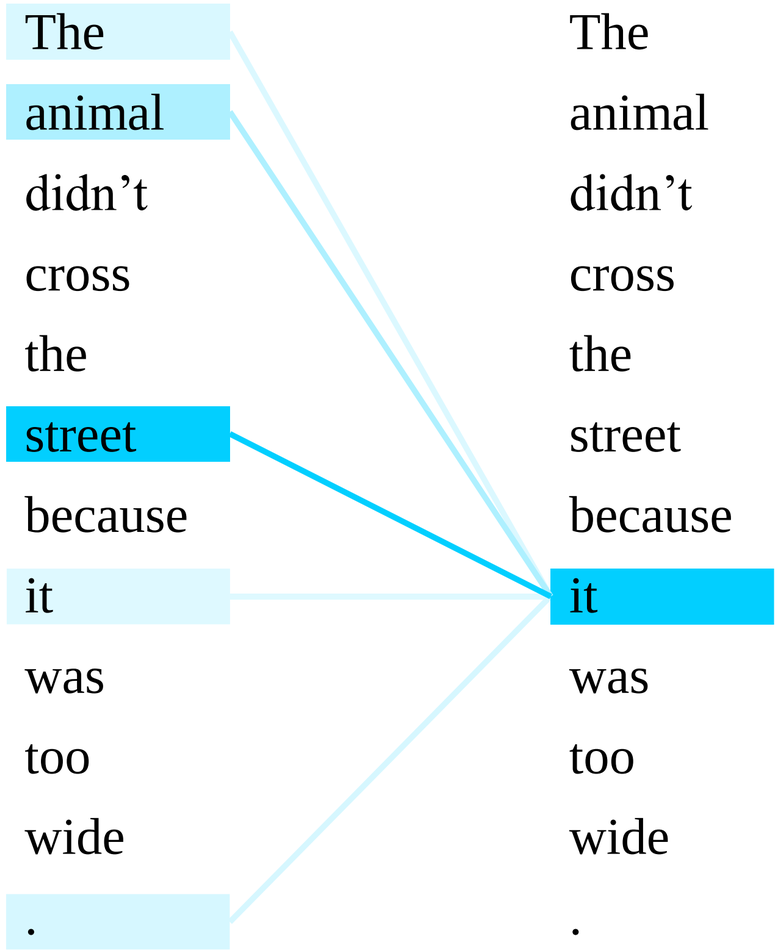
Example 1: The animal didn't cross the street because it was too tired.

Example 2: The animal didn't cross the street because it was too wide.

Transformer: Attention Example

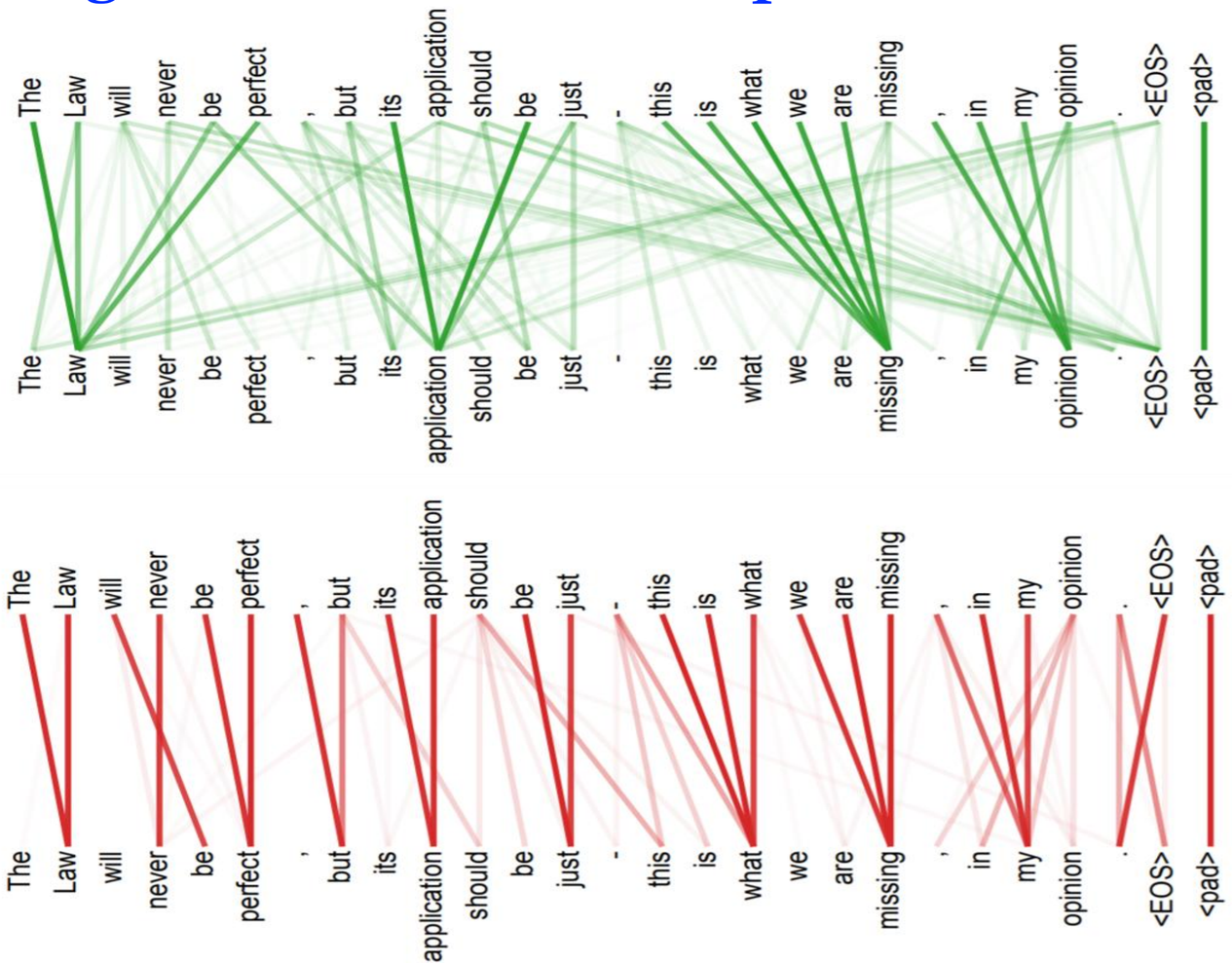
- **Below**: the encoder self-attention distribution for the word "it" from the 5th to the 6th layer of a Transformer trained on English to French translation.





Transformer: Attention Example

- Multi-head attention: red-attention about local relationship, while green-attention about long-distance relationship.

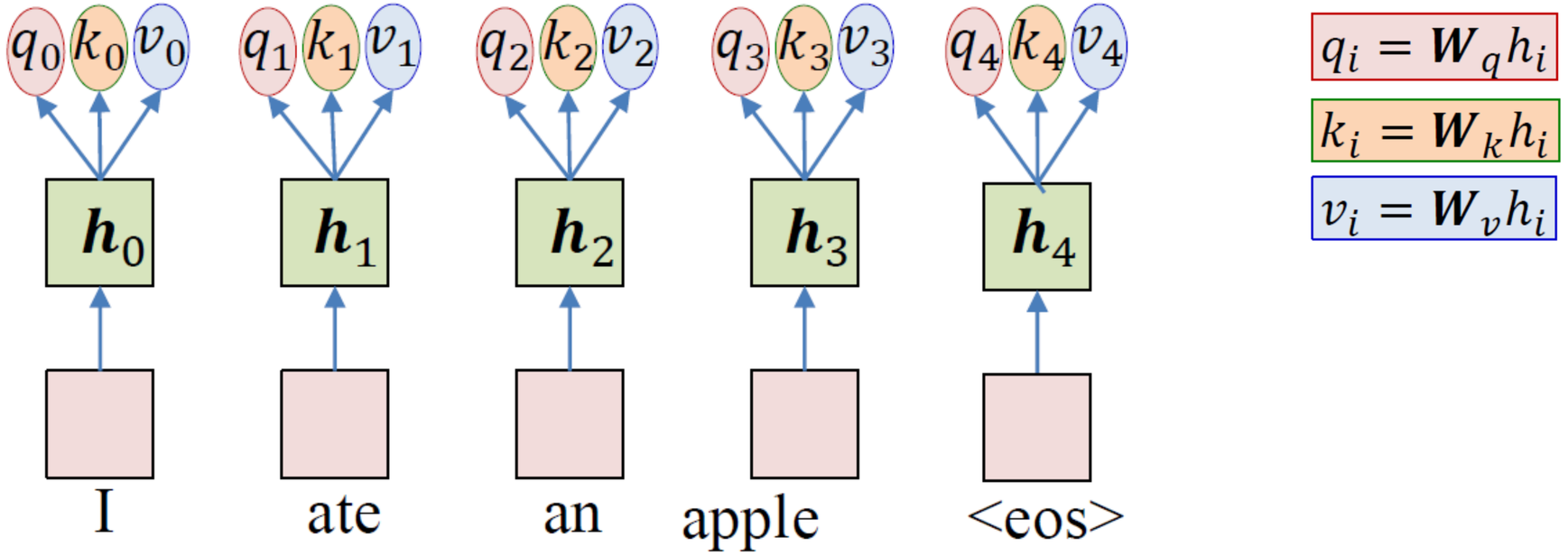


Outline

- Recap
- Self-Attention
- Position Embeddings
- Masked Self-Attention
- Transformer
- **Practice & Discussion**

Group Discussion Topic #1: Self-Attention

- Discuss within your group how Q, K, and V work together in Self-Attention, and analyze its advantages and limitations.
- One group will be randomly selected to present and answer questions.



Group Discussion Topic #1: Self-Attention

Q1: Why do we use separate Q, K, and V projections instead of the original hidden states directly?

Q2: What are the main advantages of self-attention over RNNs? Consider parallelization and long-range dependencies.

Q3: What are the main limitations of self-attention?

Group Discussion Topic #2: Multi-Head Attention

- Discuss within your group why Multi-Head Attention is needed, how it works, and its advantages and limitations.

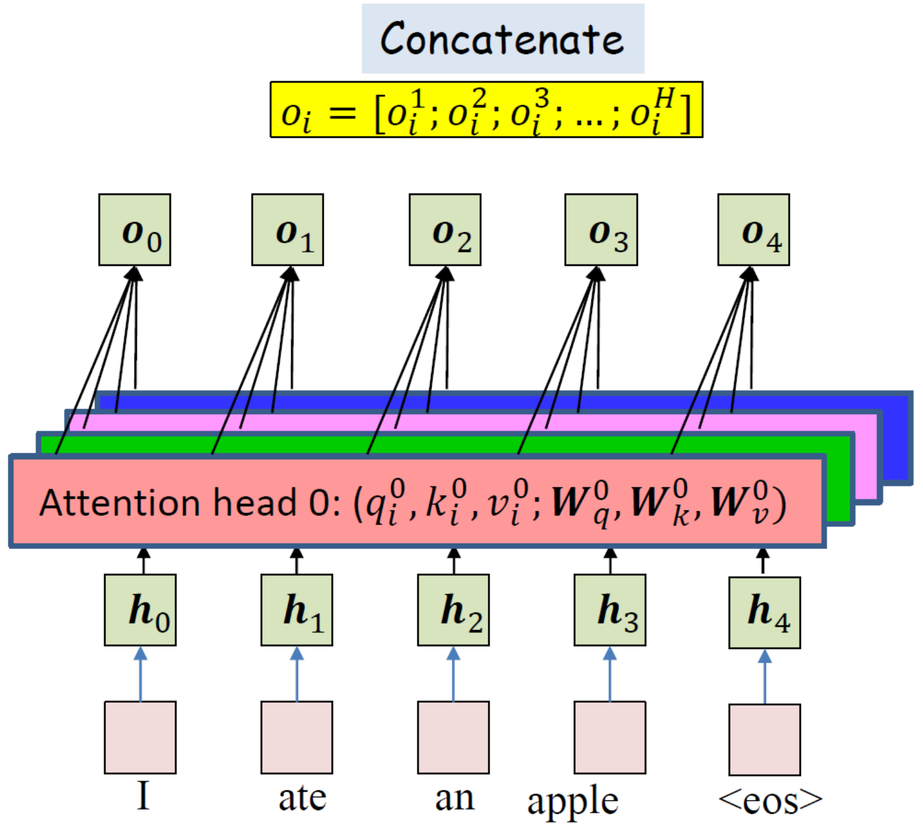
$$q_i^a = \boldsymbol{W}_q^a h_i$$

$$k_i^a = \boldsymbol{W}_k^a h_i$$

$$v_i^a = \boldsymbol{W}_v^a h_i$$

$$w_{ij}^a = attn(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$



Group Discussion Topic #2: Multi-Head Attention

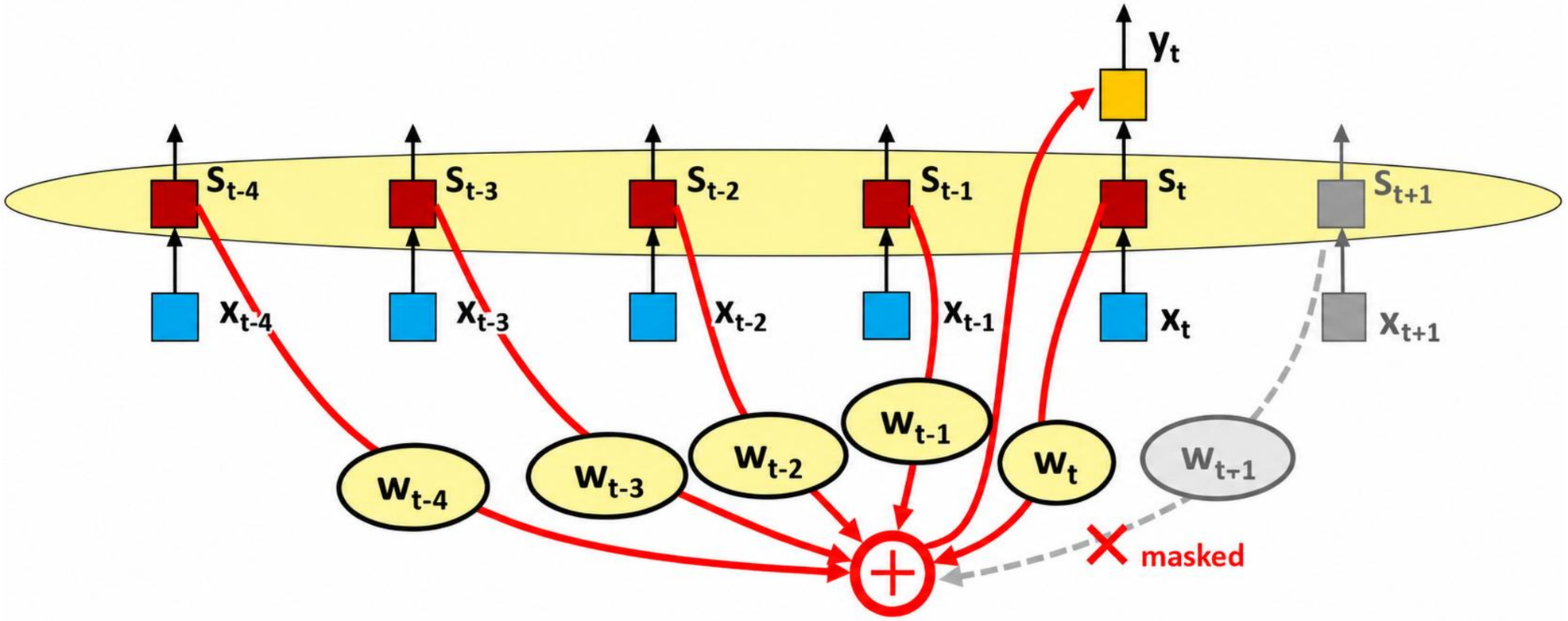
Q1: Why do we use multiple attention heads instead of a single head?

Q2: Do different attention heads necessarily learn different patterns.

Q3: What are the advantages and limitations of Multi-Head Attention?

Group Discussion Topic #3: Masked Attention

- Discuss within your group why causal masking is needed in Transformer, how it works, and its implications for training and inference.



Group Discussion Topic #3: Masked Attention

Q1: How is causal masking implemented in self-attention?

Q2: Does causal masking prevent parallel computation during training? Why or why not?

Q3: Is causal masking still necessary during autoregressive inference when KV caching is used?

Learn something useful.
Enjoy the process.